

To go fast... go slow!

[Recursion, Underlying Determinism, Strings, Cracking Polynomials]

M-x Author

INTRODUCTION TO COMPUTER SCIENCE, AY 2020/2021, SEM 2, MIDTERM PROJECT

To go fast... go slow!

[Recursion, Underlying Determinism, Strings, Cracking Polynomials]

**THIS DOCUMENT WAS ONLY MEANT TO BE VIEWED
AT THE FOLLOWING [LINK](#):**

<https://adisidev.github.io/introcs-midterm/index.html>

**THE FOLLOWING IS ONLY A DOCUMENT
REPRESENTATION FOR ARCHIVAL PURPOSES**

M-x Author

Table of Contents

1. introduction
2. recursion (recursion)
 - 2.1. introduction
 - 2.2. exercise 03
 - 2.3. exercise 04
 - 2.4. exercise 05
 - 2.4.1. Q. Compose a unit-test function for sum, based on its inductive specification.
 - 2.4.2. Q. Implement this specification as a structurally recursive function expecting a function from int to int and a non-negative integer.
 - 2.4.3. Q. Verify that this function passes your unit test.
 - 2.4.4. Q. Express your implementation using either `nat_fold_right` or `nat_parafold_right`, your choice. Justify this choice, and verify that your new implementation passes your unit test.
 - 2.4.5. Q. Re-revisit the sumtorial function (from Exercise 4) and express it using sum.
 - 2.4.6. Q. Revisit Exercise 15 in Week 03 and implement three OCaml functions that given a non-negative integer `n`, compute the sum of the `n+1` first odd numbers. The first should be structurally recursive. The second should use either `nat_fold_right` or `nat_parafold_right`, your choice. And the third should operate in constant time.
 - 2.4.7. Q. Positive Random Test
 - 2.5. exercise 06
 - 2.6. exercise 07
 - 2.7. exercise 10
 - 2.7.1. Create a unit-test function for the three predicates.
 - 2.7.2. Implement each of the three predicates independently as single recursive functions.
 - 2.7.3. Implement the three predicates as mutually recursive function
 - 2.8. conclusion
3. underlying determinism
 - 3.1. introduction
 - 3.2. question 02

- 3.3. question 03
- 3.4. question 04
 - 3.4.1. subsidiary questions
- 3.5. question 05
- 3.6. exercise 06
- 3.7. exercise 07
- 3.8. exercise 08
 - 3.8.1. subsidiary questions
- 3.9. question 09
 - 3.9.1. For any pure expression v of type `int`, would it be valid to simplify $v * 0$ into `0`?
 - 3.9.2. For any potentially impure expression e of type `int`, would it be valid to simplify $e * 0$ into `0`?
 - 3.9.3. For any pure expression v of type `int`, would it be valid to simplify $v * 1$ into v ?
 - 3.9.4. For any potentially impure expression e of type `int`, would it be valid to simplify $e * 1$ into e ?
- 3.10. exercise 10
 - 3.10.1. a. Are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):
 - 3.10.2. b. Are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):
- 3.11. exercise 11
 - 3.11.1. a. Are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):
 - 3.11.2. b. Are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):
- 3.12. exercise 12
- 3.13. conclusion
- 4. strings
 - 4.1. introduction
 - 4.2. question 01
 - 4.3. question 09
 - 4.4. question 02
 - 4.4.1. In which order are the characters accessed in the given string?
 - 4.4.2. Using a recursive function that operates over a natural number, implement a left-to-right version of `String.map`. Your implementation will therefore simulate a for-loop that goes from the smallest index to the largest index in the string.
 - 4.4.3. Using a recursive function that operates over a natural number, implement a right-to-left version of `String.map`. Your implementation will therefore simulate a for-loop that goes from the largest index to the smallest index in the string.
 - 4.5. question 03
 - 4.5.1. In which order are the characters accessed in the given string?
 - 4.5.2. Using a recursive function that operates over a natural number, implement a left-to-right version of `String.mapi`. Your implementation will therefore simulate a for-loop that goes from the smallest index to the largest index in the string.
 - 4.5.3. Using a recursive function that operates over a natural number, implement a right-to-left version of `String.mapi`. Your implementation will therefore simulate a for-loop that goes from the largest index to the smallest index in the string.
 - 4.6. question 04
 - 4.6.1. one that uses `String.mapi`,
 - 4.6.2. another that uses recursion over a non-negative integer.
 - 4.7. question 05

- 4.8. question 06
- 4.9. question 07
 - 4.9.1. Implement a palindrome detector using String.mapi.
 - 4.9.2. Implement a palindrome detector using recursion over a non-negative integer.
 - 4.9.3. Welcome Mr. Dijkstra!
- 4.10. question 08
- 4.11. question 10
- 4.12. question 11
- 4.13. conclusion
- 5. cracking polynomial functions
 - 5.1. introduction
 - 5.2. first degree
 - 5.3. second degree
 - 5.4. zeroth degree
 - 5.5. third degree
 - 5.6. fourth degree
 - 5.7. fifth degree
 - 5.8. higher degree to lower degree
 - 5.9. conclusion
- 6. conclusion

1. introduction

```
The road to wisdom? Well, it's plain  
And simple to express:  
Err  
and err  
and err again,  
but less  
and less  
and less.  
- Piet Hein
```

And that is the route this midterm will take. To make many mistakes, and to learn from them every time! What a wonderful quote!

Hang on tight, for this journey will be a long and fruitful one!

We'll start with exploring recursion (my initial plan was to link this word to the recursion sub-heading, and link the sub-heading back to this word, but that didn't quite work out; oh well!) and it's many applications. We'll learn how once we know the base case and inductive step of a problem, we can easily make a recursive program to solve it!

Next, we'll explore the underlying determinism of OCaml! In this mini-project, we'll create our own unparsers, slowly building the foundations for our very own version of OCaml. We'll learn why certain things appear to be the "norm" for OCaml when really there was no other way to go about it at all! We'll also think about the order of evaluation of various expressions and ponder upon why OCaml may have been built this way.

Subsequently, we'll look at strings (what you're reading right now!). Hence, we'll be able to look at some real world applications such as reversing a word, or checking whether a word is a palindrome! We'll even link this to the underlying determinism of OCaml and learn how we can direct OCaml to do things our way, whether our way is upside-down or downside-up!

Finally, we'll turn back to elementary algebra look at cracking polynomial functions; that is, finding the coefficients of polynomials! We'll create our own functions to find the coefficients of polynomials of various degrees and see how we can test them out!

And that will be the end of journey; we'll start on Earth and end on the Moon, and we'll have learnt so much!

2. recursion ([recursion](#))

2.1. introduction

And so we begin with recursion. And so we begin with recursion. And so we begin with recursion.

Oh no! I think we forgot the base case!

For this mini project, we begin with expressing factorial function with `nat_fold_right`, and so

begin learning about the similarity of `nat_parafold_right` and `nat_fold_right`. Next, we look at the sumtorial function *again*. Afterwards, we implement the Sigma notation and see how we may express the sumtorial function using this (and, so, we visit sumtorial, yet again). Subsequently, we look at `nat_parafold_right` and `nat_fold_right` in terms of each other, learning about the secrets of the universe (and how the two are really the same, potato potato [I suppose that doesn't work that well when you're simply reading a string]). And finally, we take one step further from the odd-even predicates - quite literally - learning about the ternary predicates, and even exploring quarternary and quinternary predicates!

I use these two definitions of `nat_fold_right` and `nat_parafold_right`, adapted from Professor Olivier Danvy's lecture notes, throughout the rest of my code.

```
(* nat_fold_right *)
let nat_fold_right zero_case succ_case n_given =
  (* nat_fold_right : 'a -> ('a -> 'a) -> int -> 'a *)
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then zero_case
    else let n' = n - 1
         in let ih = visit n'
            in succ_case ih      (* <-- succ_case takes one argument *)
  in visit n_given;;

(* nat_parafold_right *)
let nat_parafold_right zero_case succ_case n_given =
  (* nat_parafold_right : 'a -> (int -> 'a -> 'a) -> int -> 'a *)
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then zero_case
    else let n' = n - 1
         in let ih = visit n'
            in succ_case n' ih   (* <-- succ_case takes two arguments *)
  in visit n_given;;
```

I also use the following two definitions to conduct positive random tests, i.e. testing whether the function in question is the same as a witness function (that we know is correct) for a bunch of random inputs.

```
let show_bool b =
  (* show_bool : bool -> string *)
  if b
  then "true"
  else "false";;

let show_int n =
  (* show_int : int -> string *)
  if n < 0
  then "(" ^ string_of_int n ^ ")"
  else string_of_int n;;

(* positive random test *)
let positive_random_test_for_unary_function_taking_an_integer_once r name
candidate witness show_result =
  let n = Random.int r
  in let actual_result = candidate n
```

```

and expected_result = witness n
in if actual_result = expected_result
then true
else let () = Printf.printf (* error message *)
      "testing %s failed for %d failed with result %s instead of %s\n"
      name n (show_result actual_result) (show_result expected_result)
      in false;;

(* function to repeat positive random test *)
let repeat n_given test candidate =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then true
    else test candidate && visit (n - 1)
  in visit n_given;;

```

Onward to exercise 03!

2.2. exercise 03

Q. Define the factorial function using `nat_fold_right`.

Exercise 3 asks us to implement the factorial function. First, it's a good idea to have a unit test for whatever factorial function we implement!

```

(* a unit test for our future factorial function *)
let test_fac candidate =
  (* the base case: *)
  let b0 = (candidate 0 = 1)
  (* some intuitive cases: *)
  and b1 = (candidate 1 = 1)
  and b2 = (candidate 2 = 2)
  and b3 = (candidate 3 = 6)
  and b4 = (candidate 4 = 24)
  and b5 = (candidate 5 = 120)
  (* instance of the induction step: *)
  and b6 = (let n = Random.int 20
            in candidate (succ n) = (succ n) * candidate n)
  (* etc. *)
  in b0 && b1 && b2 && b3 && b4 && b5 && b6;;

```

Perfect! Now we know whether our future factorial function will horribly fail or not (mostly)! And, now to actually get on with our implementation.

Using `nat_parafold_right` may be a good start! This is because to calculate the factorial of n , we multiply n by the factorial of $(n - 1)$. Using `nat_parafold_right` will give us access to n at each stage of our computation.

```

(* factorial function using nat_parafold_right *)
let fac_npfr n =
  let () = assert (n >= 0) in
  nat_parafold_right 1 (fun n' ih -> (succ n') * ih) n;;

(* testing function with unit test *)
let () = assert (test_fac fac_npfr);;

```

And as we can see, it works!

Let's think about how it works, by trying to calculate the factorial of 5 the same way the function would:

1. To calculate the factorial of 5, we must multiply 5 by the factorial of 4. For that, we must calculate the factorial of 4.
2. To calculate the factorial of 4, we must multiply 4 by the factorial of 3. For that, we must calculate the factorial of 3.
3. To calculate the factorial of 3, we must multiply 3 by the factorial of 2. For that, we must calculate the factorial of 2.
4. To calculate the factorial of 2, we must multiply 2 by the factorial of 1. For that, we must calculate the factorial of 1.
5. To calculate the factorial of 1, we must multiply 1 by the factorial of 0. We know, because the base case is the factorial of 0, that the factorial of 0 is 1. Hence, we can pass $1 * 1$ back to the above calculation.
6. To calculate the factorial of 2, we must multiply 2 by the factorial of 1. We multiply $2 * 1$ and pass 2 to the calculation before us.
7. To calculate the factorial of 3, we must multiply 3 by the factorial of 2. We multiply $3 * 2$ and pass 6 to the calculation before us.
8. To calculate the factorial of 4, we must multiply 4 by the factorial of 3. We multiply $4 * 6$ and pass 24 to the calculation before us.
9. To calculate the factorial of 5, we must multiply 5 by the factorial of 4. We multiply $5 * 24$ and return 120.

And, the same thing is happening here!

For every run of `succ_case`, our function multiplies the current value of `n` by the factorial of `n'` (which in this case is `ih`). This happens until we reach the factorial of 0, a factorial whose value we know is 1!

Over here, we pass "1" to `nat_parafold_right` as it is the factorial of 0, and, hence, it represents our `zero_case` (that is, our base case).

Now, we can try implementing the factorial function using `nat_fold_right`, like below:

```
(* factorial function using nat_fold_right *)
let fac_nfr n =
  let () = assert (n >= 0) in
  let (n, result) =
```



```

    nat_fold_right (0, 1) (fun (n', ih) -> ((succ n'), (succ n') * ih)) n
  in result;;

(* testing function with unit test *)
let () = assert (test_fac fac_nfr);;

```

Over here, (n, result) represent the given n and the result of our calculation, and we only return the result.

We begin with (0, 1), that is our base case, because:

- 0: we must iterate from 0 until the given n
- 1: the factorial of 0 (0!), the base case, is 1

The input function simply iterates ahead through "(succ n'", and we use "(succ n') * ih" to calculate the actual factorial, (similar to how we did in our implementation with parafold) hence, by using "(succ n'" to iterate, we get access to n' as well, which we must use to calculate our factorial!

Hmmm. A language whose functions only accept only one argument where have we heard of this before? is anyone's stomach rumbling? Indian today?

A more detailed rationale of why this works has been provided in exercise 07

No really, they're the same function (we prove this later!)

Onward to exercise 4! (or, exercise 24, if you prefer).

2.3. exercise 04

Q. As a continuation of Exercise 14 in Week 03, implement two sumtorial functions: one should make as many recursive calls as the natural number it is applied to, and the other one should not even be recursive. Verify that both functions pass your unit test, and express the first using `nat_parafold_right`.

N.B.: You will need to start from scratch, i.e., with the inductive specification of the sumtorial function. (Hint: mimic the specification of the factorial function.)

Exercise 04 asks us to implement a sumtorial function in two different ways.

Before anything else, like always, lets start with a unit test:

```

(* a unit test for our future sumtorial function *)
let test_sumtorial candidate =
  (* the base case: *)
  let b0 = (candidate 0 = 0)
  (* some intuitive cases: *)
  and b1 = (candidate 1 = 1)

```

```

and b2 = (candidate 2 = 3)
and b3 = (candidate 3 = 6)
and b4 = (candidate 4 = 10)
and b5 = (candidate 5 = 15)
(* instance of the induction step: *)
and b6 = (let n = Random.int 20
          in candidate (succ n) = (succ n) + candidate n)
(* etc. *)
in b0 && b1 && b2 && b3 && b4 && b5 && b6;;

```

Let's begin with the first method: a recursive function! Maybe the following implementation will work?:

```

(* recursive function for sumtorial *)
let sumtorial_rec n =
  let () = assert (n >= 0) in
  nat_parafold_right 0 (fun n' ih -> (succ n') + ih) n;;

(* testing function with unit test *)
let () = assert (test_sumtorial sumtorial_rec);;

```

Yes! As we can see it worked.

Let's start with our implementation using `nat_parafold_right`.

The base case of the sumtorial function, i.e. the sumtorial of 0, is 0.

For every run of `succ_case`, our function adds the current value of `n` by the sumtorial of `n'` (which in this case is `ih`). This happens until we reach the sumtorial of 0, a sumtorial whose value we know is 0!

Over here, we pass "0" to `nat_parafold_right` as it is the sumtorial of 0, and, hence, it represents our `zero_case` (that is, our base case).

And now, with `nat_fold_right`:

```

(* similar to exercise 03, using nat_fold_right *)
let sumtorial_rec_nfr n =
  let () = assert (n >= 0) in
  let (n, result) =
    nat_fold_right (0, 0) (fun (n', ih) -> ((succ n'), (succ n') + ih)) n
  in result;;

(* testing function with unit test *)
let () = assert (test_sumtorial sumtorial_rec_nfr);;

```

Over here, `(n, result)` represent the given `n` and the result of our calculation, and we only return the result.

We begin with `(0, 0)`, that is our base case, because:

- 0: we must iterate from 0 until the given `n`
- 0: the sumtorial of 0, the base case, is 0

The input function simply iterates ahead through "`(succ n')`", and we use "`(succ n') + ih`" to calculate

the actual sumtorial, (similar to how we did in our implementation with parafold) hence, by using "(succ n)" to iterate, we get access to n' as well, which we must use to calculate our sumtorial!

Finally, what about a function that runs in constant time? Luckily, we have a formula for the same! Gauss's formula, of course! The formula says that the sum of all natural numbers up to n is n multiplied by n+1 divided by 2.

```
(* non-recursive function for sumtorial *)
let sumtorial_non_rec n =
  let () = assert (n >= 0) in
  n * (n + 1) / 2;; (* using Gauss's formula to calculate sum of n natural
numbers *)

(* testing function with unit test *)
let () = assert (test_sumtorial sumtorial_non_rec);;
```

And, of course, our function passes the unit test! And runs in constant time too!

We can say that it runs in constant time because it does not make more calls as n increases, like a recursive function would.

Onward to exercise $5 + 4 + 3 + 2 + 1 + 0$ (or 15, if you prefer.)

2.4. exercise 05

2.4.1.

Q

. Compose a unit-test function for sum, based on its inductive specification.

Hmm. Unit tests. We're used to those! BUT, how do we make the unit test inductive? We test the whether the function works for the inductive step or not, of course!

Let's look at the test below:

```
(* a. a unit test for our future sum function *)
let test_sum candidate f =
  (* base case *)
  let b0 = (candidate f 0 = f 0)
  (* some intuitive cases *)
  and b1 = (candidate f 1 = f 0 + f 1)
  and b2 = (candidate f 2 = f 0 + f 1 + f 2)
  (* some instances of successive steps *)
  and b3 = (candidate f 3 = candidate f 2 + f 3)
  and b4 = (candidate f 4 = candidate f 3 + f 4)
  (* instance of the induction step: *)
  and b5 = (let n = Random.int 20
            in candidate f n + f (succ n) = candidate f (succ n))
  in b0 && b1 && b2 && b3 && b4 && b5;;
```

As we can see, apart from our intuitive cases and the base case, we also have some instances where the the inductive step is tested when going from 2 to 3 and 3 to 4. Also, we test the inductive step on some random random numbers!

With the inductive step, we test whether a) applying the candidate function to n, and b) applying the candidate function to n' and adding f n to it, are the same (a = b)!

(Do note, in this case, instead of n' , I've used `succ n`. The above expression could also be written as `candidate f n' + f n = candidate f n`, where n' represents $n - 1$.)

2.4.2.

Q

. Implement this specification as a structurally recursive function expecting a function from `int` to `int` and a non-negative integer.

Let's think about what an inductive specification would look like for the sumtorial function. Perhaps, we can take inspiration from the inductive step in the unit test.

- base case: $n = 0$, we return the value yielded by applying the given function to 0.
- inductive step (getting n in terms of n'): the answer to `sum f n` in terms of n' is `f (n' + 1) + sum f n'`.

And, so we can start to implement our function recursively.

```
(* b. recursive function for sum *)
let sum f n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then f 0
    else let n' = n - 1
         in let ih = visit n'
            in f (succ n') + ih
  in visit n_given;;
```

2.4.3.

Q

. Verify that this function passes your unit test.

```
(* c. testing function with unit test *)
let () = assert (test_sum sum succ);;
```

And, of course, the function passes our unit test.

2.4.4.

Q

. Express your implementation using either `nat_fold_right` or `nat_parafold_right`, your choice. Justify this choice, and verify that your new implementation passes your unit test.

As we discussed earlier, our base case and our inductive step are:

- base case: $n = 0$, we return the value yielded by applying the given function to 0.
- inductive step (getting n in terms of n'): the answer to `sum f n` in terms of n' is `f (n' + 1) + sum f n'`.

Hence, we see that our inductive step, which is implemented by our `succ_case` function, requires us to know what `n` or `n'` is.

Hence, it makes more sense for us to use `nat_parafold_right` rather than `nat_fold_right`.

Hence, we can substitute our `zero_case` (that is, our base case) and our `succ_case` function in `nat_parafold_right` as below:

```
(* d. recursive function for sum using nat_parafold_right *)
let sum_npfr f n =
  nat_parafold_right (f 0) (fun n' ih -> f (succ n') + ih) n;;

(* testing function with unit test *)
let () = assert (test_sum sum_npfr (fun x -> 2 * x));;
```

And, of course, the function passes our unit test.

That being said, we can still implement the function with `nat_fold_right` as well (later we'll see how `nat_fold_right` and `nat_parafold_right` are really the same, stay tuned!):

```
(* similar to exercise 03, using nat_fold_right *)
let sum_nfr f n =
  let (n, result) =
    nat_fold_right (0, (f 0)) (fun (n', ih) -> (succ n', (f (succ n') + ih))) n
  in result;;

(* testing function with unit test *)
let () = assert (test_sum sum_nfr (fun x -> 2 * x));;
```

Over here, `(n, result)` represent the given `n` and the result of our calculation, and we only return the result.

We begin with `(0, (f 0))`, that is our base case, because:

- 0: we must iterate from 0 until the given `n`
- 0: for our base case, we return the result of applying the given function to 0

The input function simply iterates ahead through "`(succ n')`", and we use "`f (succ n') + ih`" to calculate the subsequent result (similar to how we did in our implementation with `parafold`). Hence, by using "`(succ n')`" to iterate, we get access to `n'` as well, which we must use to calculate our result!

And, of course, the function passes our unit test.

2.4.5.

Q

. Re-revisit the sumtorial function (from Exercise 4) and express it using sum.

For the sumtorial function, we simply need to add all numbers up to `n`. Hence, we don't need to

apply any function to the numbers up to n ; we can simply use the identity function!

```
(* e. representing sumtorial using sum *)
let sumtorial sum n =
  sum_nfr (fun x -> x) n;;

(* testing function with unit test *)
let () = assert (test_sumtorial sumtorial_sum);;
```

Our implementation of sumtorial using sum now becomes much easier!

And, of course, the function passes our unit test.

2.4.6.

Q

. Revisit Exercise 15 in Week 03 and implement three OCaml functions that given a non-negative integer n , compute the sum of the $n+1$ first odd numbers. The first should be structurally recursive. The second should use either `nat_fold_right` or `nat_parafold_right`, your choice. And the third should operate in constant time.

Before anything else, like always let's first write a unit test so we can test (to some extent) whether our function does not work! **(I intentionally did not say that we can test whether the function does work... more on this later!)**

```
(* a unit test for our future oddsum function *)
let test_oddsum candidate =
  (* the base case: *)
  let b0 = (candidate 0 = 1)
  (* some intuitive cases: *)
  and b1 = (candidate 1 = 4)
  and b2 = (candidate 2 = 9)
  and b3 = (candidate 3 = 16)
  and b4 = (candidate 4 = 25)
  and b5 = (candidate 5 = 36)
  (* instance of the induction step: *)
  and b6 = (let n = Random.int 20
            in candidate (succ n) = (make_odd_number (succ n)) + candidate (n))
  (* etc. *)
  in b0 && b1 && b2 && b3 && b4 && b5 && b6;;
```

Perfect! Now we can actually go ahead and try implementing our function!

Perhaps, what may be helpful is a function that maps a number n to the $n+1$ th odd number.

```
(* a function to make a number odd *)
let make_odd_number n =
  let () = assert (n >= 0)
  in 2 * n + 1;;
```

This function takes a number, n , and maps it to the $n+1$ th odd number.

Any number multiplied by 2 results in an even number. Adding 1 to an even number results in an odd number!

Hence, this function multiplies the number by 2 to make it even and then subsequently adds 1 to make it odd.

Since every alternate number is odd, we multiply n by 2. We then add 1 to make the number odd. Hence, this function maps n to the $n+1$ th odd number.

Great! Now that we have that settled we can actually work on the implementation itself. Perhaps we could first try implementing it with the sum function we just implemented.

So, the sum function applies a function to every number up till n and then adds the results together. The function we're looking for is one that maps a number, n , to the $n+1$ th number. Sound familiar? It's the function we just implemented! Let's try using that!

```
(* using sum *)
let oddsum_sum_v0 n =
  sum (fun i -> 2 * i + 1) n;;

(* testing function with unit test *)
let () = assert (test_oddsum oddsum_sum_v0);;
```

It doesn't not work! That is to say, the function passes the unit test!

Let's put it slightly more succinctly:

```
(* more succinctly *)
let oddsum_sum_v1 n =
  sum make_odd_number n;;

(* testing function with unit test *)
let () = assert (test_oddsum oddsum_sum_v1);;
```

And, of course, again, the function passes our unit test.

Now, let's try something that is structurally recursive, perhaps taking inspiration from the structurally recursive version of the sum function?

```
(* structurally recursive *)
let oddsum_rec n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then 1
    else let n' = n - 1
         in let ih = visit n'
         in ih + (make_odd_number n)
  in visit n_given;;

(* testing function with unit test *)
let () = assert (test_oddsum oddsum_rec);;
```

Hmmm. So, let's first *visit* (pun intended, hehe) our base case.

The base case, i.e. the sum of the first 0+1 (or, more commonly known as, 1) odd numbers is 0. And so we have our base case.

Next, let's look at the induction step. In our sum function, the induction step was simply applying

the given function to $n' + 1$ and adding ih to it! Aaand... the given function here is the function to map a number n to the $n+1$ th odd number; so why don't we just use that again?

And, of course, the function passes our unit test.

Let's now think about our implementation with `nat_parafold_right`.

The base case as just mentioned 1.

For every run of `succ_case`, our function adds the result of applying `make_odd_number` to $n+1$ to result of applying `make_odd_number` to n' (which in this case is ih). This happens until we reach the 0, the result of which we know is 1!

Over here, we pass "0" to `nat_parafold_right` as it is the sumtorial of 0, and, hence, it represents our `zero_case` (that is, our base case).

```
(* using nat_parafold_right *)
let oddsum_npfr n =
  nat_parafold_right 1 (fun n' ih -> ih + (make_odd_number (succ n')))) n;;

(* testing function with unit test *)
let () = assert (test_oddsum oddsum_npfr);;
```

And, of course, the function passes our unit test.

Finally, as mentioned a few times above, the same can be implemented using `nat_fold_right`!

```
(* similar to exercise 03, using nat_fold_right *)
let oddsum_nfr n =
  let (n, result) =
    nat_fold_right (0, 1) (fun (n', ih) -> (succ n', (ih + (make_odd_number
(succ n'))))) n
  in result;;

(* testing function with unit test *)
let () = assert (test_oddsum oddsum_nfr);;
```

Over here, $(n, result)$ represent the given n and the result of our calculation, and we only return the result.

We begin with $(0, 1)$, that is our base case, because:

- 0: we must iterate from 0 until the given n
- 1: the sum of the first odd number, 1, is 1

The input function simply iterates ahead through `(succ n')`, and we use `(make_odd_number (succ n')) + ih` to calculate the actual result, (similar to how we did in our implementation with `parafold`) hence, by using `"(succ n)"` to iterate, we get access to n' as well, which we must use to calculate our result!

And, of course, the function passes our unit test.

Wait a second, are we not forgetting something? Well, of course: the function that runs in constant time! The holy grail!

Well we already know what the sum of n odd numbers is in terms of n : $n * n$. Hence, the sum of $n+1$ odd numbers is $(n + 1) * (n + 1)$, which we can use to calculate the answer in constant time!

```
(* constant time *)
let oddsum_const_v0 n =
  let () = assert (n >= 0) in
  (succ n) * (succ n);;

(* testing function with unit test *)
let () = assert (test_oddsum oddsum_const_v0);;
```

And, of course, the function passes our unit test.

```
(* lesser calculations, more space *)
let oddsum_const_v1 n =
  let () = assert (n >= 0) in
  let n' = succ n in
  n' * n';;

(* testing function with unit test *)
let () = assert (test_oddsum oddsum_const_v1);;
```

Over here, `succ n` does not need to be calculated twice. However, we store `n'` in memory (in the local environment of `n' * n'`). Hence, this version takes more space but less calculations!

2.4.7. . Positive Random Test

Q

We have one final trick up our sleeves that we haven't used until now: positive random tests! We can use the constant function, for which we have a mathematical proof, as a witness function! That is, a function to test our other implementations against!

```
let positive_random_test_oddsum_once r name candidate =
  positive_random_test_for_unary_function_taking_an_integer_once
    r name candidate oddsum_const_v1 show_int;;

let () = assert (repeat 100 (positive_random_test_oddsum_once 100 "oddsum_npfr")
  oddsum_npfr);;
let () = assert (repeat 100 (positive_random_test_oddsum_once 100 "oddsum_nfr")
  oddsum_nfr);;
let () = assert (repeat 100 (positive_random_test_oddsum_once 100
  "oddsum_sum_v1") oddsum_sum_v1);;
let () = assert (repeat 100 (positive_random_test_oddsum_once 100 "oddsum_rec")
  oddsum_rec);;

(* let () = assert (repeat 100 (positive_random_test_oddsum_once 100
  "fake_function") (fun i -> i));; *)
(* fails test, with output:
  testing fake_function failed for 9 failed with result 9 instead of 100 *)
```

And, as we can see all our previous implementations still pass multiple runs of the positive random

test! Further, a fake function, `fake_function`, fails and gives a helpful error message!

2.5. exercise 06

Q. Implement `nat_fold_right` in terms of `nat_parafold_right`:

Well, this is really quite easy! Even if our function accepts "multiple arguments", we don't really need to use ALL (or really any, as we'll see later) of them.

The `succ_case` function of `nat_parafold_right` takes accepts two arguments. On the other hand, the `succ_case` function of `nat_fold_right` takes only one argument.

Hence, we can simply not use the second argument of the `succ_case` function of `nat_parafold_right` to implement `nat_fold_right`!

Behold:

```
let nat_fold_right_using_parafold zero_case succ_case n =  
  nat_parafold_right zero_case (fun n' ih -> succ_case ih) n;;
```

Hmmm. Let's also try testing it by comparing it to our original `nat_fold_right` function, that can act as a witness function!

```
(* a function to test our new nat_fold_right function *)  
let test_nat_fold_right zero_case succ_case =  
  let n = Random.int 20  
  in nat_fold_right zero_case succ_case n = nat_fold_right_using_parafold  
  zero_case succ_case n;;  
  
(* testing function with testing function *)  
let () = assert (test_nat_fold_right 0 (fun ih -> ih + 2));;
```

Good enough? Good enough!

2.6. exercise 07

Now, this one is a slightly tougher cookie!

The only difference between `nat_fold_right` and `nat_parafold_right` is that `nat_parafold_right`'s function that calculates the successive case, `succ_case`, takes, ahem, "2" arguments whereas `nat_fold_right`'s `succ_case` only accepts one argument.

But, wait a second [name redacted], doesn't any function in OCaml accept only one argument? YES! And there comes in the "difference" of the curried and the uncurried function.

What a "function with multiple arguments" actually is, is simply a function that takes a function that takes a variable of a certain type, an integer in this case.

And that is what we call the curried function!

There is another way we can pass in "multiple arguments" without implementing a curried function: we pass tuples!

Tuples can contain multiple values that are of any type. And that is exactly what we do below, when we pass a pair of `(int * int)`, that is we define the `succ_case` function of `nat_fold_right` such that it accepts a pair.

And this is what we call the uncurried function!

```
let nat_parafold_right_using_fold zero_case succ_case n =
  let (n, result) =
    nat_fold_right (0, zero_case) (fun (n', ih) -> ((succ n'), succ_case n' ih))
  n
  in result;;

(* a function to test our new nat_parafold_right function *)
let test_nat_parafold_right zero_case succ_case =
  let n = Random.int 20
  in nat_parafold_right zero_case succ_case n = nat_parafold_right_using_fold
  zero_case succ_case n;;

(* testing function with testing function *)
let () = assert (test_nat_parafold_right 1 (fun n' ih -> (succ n') * ih));;
```

Huzzah!

A few notes:

the pair `(n, result)` is named so because:

- `n`: the final value of this will be `n` as we would have iterated from 0 up till `n`
- `result`: the final value of this will be the result depending on the `succ_case` function

we pass in `(0, zero_case)` to `nat_parafold_right` because:

- `0`: we begin with 0 and iterate up till `n`
- `zero_case`: this is the base case of the what we aim to achieve.
this is what the `succ_case` function will start building upon!

And finallyy... as we saw in question 04 of the midterm project about the underlying determinism of OCaml, the curried function and the uncurried function are equivalent, that is: they give rise to the same evaluation/computation!

And since the only difference between `nat_fold_right` here and `nat_parafold_right` here is the use of an uncurried function and the use of a curried function, `nat_fold_right` and `nat_parafold_right` are equivalent!

```
"hmmm. a language whose functions only accept only one argument
where have we heard of this before?
is anyone's stomach rumbling? Indian today maybe?"
- [name redacted], 2021
```

2.7. exercise 10

Hang on tight, this one is going to be a long one.

2.7.1.

reate a unit-test function for the three predicates.

C

Yes! A unit test! Observe:

```
(* a unit test for our future ternary predicates *)
let test_ternary candidate n =
  (* pass n as 0 for ternary, 1 for post-ternary, 2 for pre-ternary *)
  (* the base case: *)
  let b0 = (candidate 0 = ((0 mod 3) = n))
  and b1 = (candidate 1 = ((1 mod 3) = n))
  and b2 = (candidate 2 = ((2 mod 3) = n))
  and b3 = (candidate 3 = ((3 mod 3) = n))
  and b4 = (candidate 4 = ((4 mod 3) = n))
  and b5 = (candidate 5 = ((5 mod 3) = n))
  (* instance of a truth that should hold: *)
  and b6 = (let i = Random.int 20
            in candidate i = candidate (i + 3))
  (* etc. *)
  in b0 && b1 && b2 && b3 && b4 && b5 && b6;;
```

I decided to kill 3 birds with one stone (something that will happen once again) and implemented the unit test in a way that one unit test, with a certain input, can account for all the three predicates! A number mod 3 results in either 0, 1 or 2.

- 0 means that the number perfectly divided by 3, and is, hence, ternary.
- 1 means that the number - 1 would divide perfectly and hence, it is post-ternary (one too much).
- 2 means that the number + 1 would divide perfectly, and hence, it is pre-ternary (one too less).

Hence, by passing what the result of the mod calculation above would be to the unit test, we get all 3 unit tests bundled into one!

2.7.2.

implement each of the three predicates independently as single recursive functions.

I

Next, let's try actually implementing the predicates!

If we are able to implement the ternary predicate, the post and pre ternary predicates should fall just in place!

Perhaps, we could recursively subtract (similar to division) 3 from a number until it becomes lesser than 3.

Once it is lesser than 3, if we're left with 0, the number was able to perfectly divide with 3, and hence, it passes the ternary predicate! If we're left with anything else, the number was not ternary.

```
let ternary_v0 n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then true
    else if n < 3
    then false
    else let n' = n - 3
         in let ih = visit n'
         in ih
  in visit n_given;;

(* testing function with unit test *)
let () = assert (test_ternary ternary_v0 0);;
```

And, of course, the function passes our unit test.

What about pre-ternary numbers? Well, once the number is less than 3, if we're left with 2, then the number was pre-ternary! If we're left with anything else, the number was not pre-ternary.

And so we have:

```
let pre_ternary_v0 n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 2
    then true
    else if n < 3
    then false
    else let n' = n - 3
         in let ih = visit n'
         in ih
  in visit n_given;;

(* testing function with unit test *)
let () = assert (test_ternary pre_ternary_v0 2);;
```

And, of course, the function passes our unit test.

Finally, we have post-ternary. Applying the same line of reasoning, once the number is less than 3, if we're left with 1, then number was post-ternary! If we're left with anything else, the number was not post-ternary.

And behold:

```

let post_ternary_v0 n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 1
    then true
    else if n < 3
    then false
    else let n' = n - 3
         in let ih = visit n'
            in ih
  in visit n_given;;

(* testing function with unit test *)
let () = assert (test_ternary post_ternary_v0 1);;

```

If we observe closely, everything in the function stays the same apart from the what we compare the number we're left with to. Hence, we can parameterize that, and, kill 3 birds with one stone (once again)!

```

(* alternatively, to kill 3 birds with 1 stone *)
let ternary_predicates base_case n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = base_case
    then true
    else if n < 3
    then false
    else let n' = n - 3
         in let ih = visit n'
            in ih
  in visit n_given;;

let ternary_v1 n =
  ternary_predicates 0 n;;

(* testing function with unit test *)
let () = assert (test_ternary ternary_v1 0);;

let pre_ternary_v1 n =
  ternary_predicates 2 n;;

(* testing function with unit test *)
let () = assert (test_ternary pre_ternary_v1 2);;

let post_ternary_v1 n =
  ternary_predicates 1 n;;

(* testing function with unit test *)
let () = assert (test_ternary post_ternary_v1 1);;

```

And, of course, all our implementations pass the unit test!

However, the above implementations do not use the inductive specification of ternary! Therefore, the usage of "ih" seems to be a misnomer, as it effectively skips various inductive steps and can no longer be said to be truly inductively defined.

Back to the drawing board!

Well let's begin with the ternary predicate. Let's think about what an inductive specification would look like for the ternary predicate.

- base case: $n = 0$, we know 0 is a ternary number!
- inductive step: well, if n' is 0, then n cannot be a ternary number, as 1 a post-ternary number!
- inductive step after inductive step: well, if n'' (that is $n' - 1$) is 0, then n cannot be a ternary number, as 2 is a pre-ternary number!
- inductive step after inductive step after inductive step: well, now n must be whatever n''' is, as n and $n - 3$ should both satisfy the same predicate!

And behold:

```
let ternary_v2 n_given =  
  let () = assert (n_given >= 0) in  
  let rec visit n =  
    if n = 0  
    then true  
    else let n' = pred n  
         in if n' = 0  
            then false  
            else let n'' = pred n'  
                 in if n'' = 0  
                    then false  
                    else let n''' = pred n''  
                         in visit n'''  
  in visit n_given;;  
  
(* testing function with unit test *)  
let () = assert (test_ternary ternary_v2 0);;
```

And similarly, we have pre-ternary and post-ternary:

For the pre-ternary predicate:

- base case: $n = 0$, we know 0 is not a pre-ternary number!
- inductive step: well, if n' is 0, then n cannot be a pre-ternary number, as 1 a post-ternary number!
- inductive step after inductive step: well, if n'' (that is $n' - 1$) is 0, then n must be pre-ternary number as 2 is a pre-ternary number!
- inductive step after inductive step after inductive step: well, now n must be whatever n''' is, as n and $n - 3$ should both satisfy the same predicate!

Hurrah:

```
let pre_ternary_v2 n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then false
    else let n' = pred n
         in if n' = 0
            then false
            else let n'' = pred n'
                 in if n'' = 0
                    then true
                    else let n''' = pred n''
                         in visit n'''
  in visit n_given;;

(* testing function with unit test *)
let () = assert (test_ternary pre_ternary_v2 2);;
```

And, finally, post-ternary:

- base case: $n = 0$, we know 0 is not a post-ternary number!
- inductive step: well, if n' is 0, then n must be a post-ternary number, as 1 a post-ternary number!
- inductive step after inductive step: well, if n'' (that is $n' - 1$) is 0, then cannot be a post-ternary number as 2 is a pre-ternary number!
- inductive step after inductive step after inductive step: well, now n must be whatever n''' is, as n and $n - 3$ should both satisfy the same predicate!

[insert exclamation]:

```
let post_ternary_v2 n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then false
    else let n' = pred n
         in if n' = 0
            then true
            else let n'' = pred n'
                 in if n'' = 0
                    then false
                    else let n''' = pred n''
                         in visit n'''
  in visit n_given;;

(* testing function with unit test *)
let () = assert (test_ternary post_ternary_v2 1);;
```

And all of our implementations passed our unit tests!
BUT lets go a little more extreme: positive random tests!

1. Positive Random Tests

Let's first begin by creating our own witness functions:

```
(* a witness function for the ternary predicate *)
let ternary_witness n =
  n mod 3 = 0;;

(* similarly, a witness function for pre-ternary and post-ternary *)
let pre_ternary_witness n =
  n mod 3 = 2;;

let post_ternary_witness n =
  n mod 3 = 1;;
```

Now, we can test our implementations against these witness tests!

```
(* we can now use these witness functions to test
   our other implementations! *)
let positive_random_test_ternary_once r name candidate =
  positive_random_test_for_unary_function_taking_an_integer_once
    r name candidate ternary_witness show_bool;;

let positive_random_test_preternary_once r name candidate =
  positive_random_test_for_unary_function_taking_an_integer_once
    r name candidate pre_ternary_witness show_bool;;

let positive_random_test_postternary_once r name candidate =
  positive_random_test_for_unary_function_taking_an_integer_once
    r name candidate post_ternary_witness show_bool;;
```

And, finally, running the tests repeatedly:

```
let () = assert (repeat 100 (positive_random_test_ternary_once 100 "ternary_v2")
  ternary_v2);;
let () = assert (repeat 100 (positive_random_test_preternary_once 100
  "pre_ternary_v2") pre_ternary_v2);;
let () = assert (repeat 100 (positive_random_test_postternary_once 100
  "post_ternary_v2") post_ternary_v2);;
```

2. Quarternary Recursively

Similar to ternary, we can also define the quarternary predicates recursively.

For the quarternary predicate:

- base case: $n = 0$, we know 0 is a quarternary number!
- inductive step: well, if n' is 0, then n cannot be a quarternary number, as 1 a post-quarternary number!
- inductive step after inductive step: well, if n'' (that is $n' - 1$) is 0, then n cannot be a quarternary number, as 2 is a mid-quarternary number!

- inductive step after inductive step after inductive step: if n''' is 0, then n cannot be a quaternary number, as 3 is a pre-quaternary number!
- inductive step after inductive step after inductive step after inductive step: well, now n must be whatever n'''' is, as n and $n - 4$ should both satisfy the same predicate!

With similar reasoning for pre-quaternary, mid-quaternary, post-quaternary:

```
(* we can also recursively define quaternary *)
(* quaternary *)
let quaternary n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then true
    else let n' = pred n
         in if n' = 0
            then false
            else let n'' = pred n'
                 in if n'' = 0
                    then false
                    else let n''' = pred n''
                         in if n''' = 0
                            then false
                            else let n'''' = pred n'''
                                 in visit n''''
  in visit n_given;;

let mid_quaternary n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then false
    else let n' = pred n
         in if n' = 0
            then false
            else let n'' = pred n'
                 in if n'' = 0
                    then true
                    else let n''' = pred n''
                         in if n''' = 0
                            then false
                            else let n'''' = pred n'''
                                 in visit n''''
  in visit n_given;;

let pre_quaternary n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then false
    else let n' = pred n
         in if n' = 0
            then false
            else let n'' = pred n'
                 in if n'' = 0
                    then true
                    else let n''' = pred n''
                         in if n''' = 0
                            then false
                            else let n'''' = pred n'''
                                 in visit n''''
  in visit n_given;;
```

```

                                then true
                                else let n''' = pred n''
                                      in visit n'''

in visit n_given;;

let post_quarternary n_given =
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then false
    else let n' = pred n
          in if n' = 0
              then true
              else let n'' = pred n'
                    in if n'' = 0
                        then true
                        else let n''' = pred n''
                              in if n''' = 0
                                  then false
                                  else let n'''' = pred n'''
                                        in visit n''''
          in visit n'
  in visit n
in visit n_given;;

```

3. Using nat_fold_right

Finally, we can also define ternary using nat_fold_right!

Let's first implement a function that cycles a triple:

```

(* function to cycle triple *)
let tricycle (b1, b2, b3) =
  (b3, b1, b2);;

```

Now, for implementing the ternary predicate using nat_fold_right, the succ_case function would simply cycle through a triple of booleans.

The booleans would have the base case (true, false, false) as 0 is ternary, 1 post-ternary and 2 pre-ternary! Whatever position the true bool is, that is the kind of predicate that the number satisfies!

For example, if we cycle the triple thrice, we end up with the same triple we began with. Since true is in the first position, the number is ternary (as 3 is ternary)!

If true were in the second position twice, the triple cycled one too many times, and hence, the number is post-ternary!

Finally, if the true were in third place, the triple cycled one too few times, and hence, the number is pre-ternary!

And so we have:

```

(* through nat_fold_right *)
let all_ternary_v3 n =
  nat_fold_right (true, false, false) tricycle n;;

let ternary_v3 n =
  let (b1, b2, b3) = all_ternary_v3 n
  in b1;;

let pre_ternary_v3 n =

```

```

let (b1, b2, b3) = all_ternary_v3 n
in b3;;

let post_ternary_v3 n =
  let (b1, b2, b3) = all_ternary_v3 n
  in b2;;

```

And all of these can be tested with a single unit test:

```

(* all our functions can now be tested with a single unit test *)
let test_all_ternary candidate =
  (* the base case: *)
  let b0 = (candidate 0 = (true, false, false)) (* ternary *)
  and b1 = (candidate 1 = (false, true, false)) (* post-ternary *)
  and b2 = (candidate 2 = (false, false, true)) (* pre-ternary *)
  and b3 = (candidate 3 = (true, false, false)) (* ternary *)
  and b4 = (candidate 4 = (false, true, false)) (* post-ternary *)
  and b5 = (candidate 5 = (false, false, true)) (* pre-ternary *)
  (* instance of the induction step: *)
  and b6 = (let i = Random.int 20
            in candidate i = candidate (i + 3))
  (* etc. *)
  in b0 && b1 && b2 && b3 && b4 && b5 && b6;;

(* testing function(s) with unit test *)
let () = assert (test_all_ternary all_ternary_v3);;

```

We can also try our positive random tests on these:

```

(* we can also run our positive random test on these functions *)
let () = assert (repeat 100 (positive_random_test_ternary_once 100 "ternary_v3")
  ternary_v3);;
let () = assert (repeat 100 (positive_random_test_preternary_once 100
  "pre_ternary_v3") pre_ternary_v3);;
let () = assert (repeat 100 (positive_random_test_postternary_once 100
  "post_ternary_v3") post_ternary_v3);;

```

And, finally, similarly for quaternary and quinary!

```

(* we can also implement a similar function for quaternary numbers *)

(* function to cycle quadruple *)
let quadcycle (b1, b2, b3, b4) =
  (b4, b1, b2, b3);;

let all_quaternary_v1 n =
  nat_fold_right (true, false, false, false) quadcycle n;;

let quaternary_v1 n =
  let (b1, b2, b3, b4) = all_quaternary_v1 n
  in b1;;

let mid_quaternary_v1 n =
  let (b1, b2, b3, b4) = all_quaternary_v1 n
  in b3;;

let pre_quaternary_v1 n =
  let (b1, b2, b3, b4) = all_quaternary_v1 n
  in b4;;

```

```

let post_quarternary_v1 n =
  let (b1, b2, b3, b4) = all_quarternary_v1 n
  in b2;;

(* similarly, quinternary *)

(* function to cycle quintuple *)
let quintcycle (b1, b2, b3, b4, b5) =
  (b5, b1, b2, b3, b4) ;;

let all_quinternary n =
  nat_fold_right (true, false, false, false, false) quintcycle n;;

```

2.7.3. **implement the three predicates as mutually recursive function**

Let's think about how we would implement the ternary predicates as mutually recursive functions!

Well, let's first imagine a scenario where we call the ternary predicate on a number, n :

- if the n is 0, we can return true as we know that 0 is a ternary number!
- however, if the number is not 0, we can simply test whether $n-1$ is a pre-ternary number! if $n-1$ is a pre-ternary number, then n must be a ternary number! so, we can simply call the pre-ternary predicate on $n-1$!
- now, for calling the pre-ternary predicate on $n-1$: if $n-1$ is 0, we can return false, as 0 is not a pre-ternary number, and, therefore, n cannot be a ternary number, as 1 is not a ternary number! however, if $n-1$ is not 0, we can simply test whether $n-2$ is a post-ternary number! if $n-2$ is a post-ternary number, $n-1$ must be a pre-ternary number, and n must be a ternary number! hence, we can simply call the post-ternary predicate on $n-2$
- now, for calling the post-ternary predicate on $n-2$: if $n-2$ is 0, we can return false, as 0 is not a post-ternary number, and, therefore, n cannot be a ternary number, as 2 is not a ternary number! however, if $n-2$ is not 0, we can simply test whether $n-3$ is a ternary number! if $n-3$ is a ternary number, $n-2$ must be a post-ternary number, $n-1$ must be a pre-ternary number, and n must be a ternary number! hence, we can simply call the ternary predicate on $n-3$

Hence, we arrive at a mutually recursive definition for the ternary predicates:

```

(* ternary predicate mutually recursively *)
let rec ternary_v4 n =
  if n = 0
  then true
  else let n' = n - 1
       in pre_ternary_v4 n'
and pre_ternary_v4 n =
  if n = 0
  then false
  else let n' = n - 1

```

```

        in post_ternary_v4 n'
and post_ternary_v4 n =
  if n = 0
  then false
  else let n' = n - 1
        in ternary_v4 n';;

(* testing function with unit test *)
let () = assert (test_ternary ternary_v4 0);;

(* testing function with unit test *)
let () = assert (test_ternary pre_ternary_v4 2);;

(* testing function with unit test *)
let () = assert (test_ternary post_ternary_v4 1);;

(* we can also run our positive random test on these functions *)
let () = assert (repeat 100 (positive_random_test_ternary_once 100 "ternary_v4")
ternary_v4);;
let () = assert (repeat 100 (positive_random_test_preternary_once 100
"pre_ternary_v4") pre_ternary_v4);;
let () = assert (repeat 100 (positive_random_test_postternary_once 100
"post_ternary_v4") post_ternary_v4);;

```

And, of course, the function passes our unit test. Further, they also pass the positive random tests when tested against the witness functions!

Finally, we can similarly define the quarternary predicates mutually recursively:

```

(* and, of course, the quarternary predicate as well, mutually recursively *)
let rec quarternary_v2 n =
  if n = 0
  then true
  else let n' = n - 1
        in pre_quarternary_v2 n'
and pre_quarternary_v2 n =
  if n = 0
  then false
  else let n' = n - 1
        in post_quarternary_v2 n'
and mid_quaternary_v2 n =
  if n = 0
  then false
  else let n' = n - 1
        in post_quarternary_v2 n'
and post_quarternary_v2 n =
  if n = 0
  then false
  else let n' = n - 1
        in quarternary_v2 n';;

```

2.8. conclusion

And there we have it! A miscellany of recursive programs, completed! Wow, that was some journey!

We learnt so many things on the way; we learnt how `nat_parafold_right` and `nat_fold_right` are really the same function (which we'll learn even more about in the next

section, when we talk about curried and uncurried functions). We also made a few mistakes along the way, where what we thought initially looked like it was inductively defined was not really inductively defined; but we were able to fix those mistakes! We learnt how things scale, by not only building the ternary predicates, but also the quaternary and quinternary predicates! We also saw the connection between the sumtorial and the sigma function. We also saw how the definition of factorial is recursive in nature! And, again, we learnt from our mistakes! Onward!

3. underlying determinism

3.1. introduction

OCaml makes many decisions for us before we even start programming; throughout this section we look at some of these decisions and unravel the mystery behind them. We study the order in which function applications are evaluated: function first or argument first? We study why it may be one way or the other. We study in what order are tuples created. We learn about the curried variety and the uncurried variety of functions, neither tastier than the other (as they're just the same)! We differentiate between function applications and let expressions and the order in which let expressions are evaluated; simultaneously, we learn about the order in which definiens are evaluated in a let and expression. We also learn about boolean conjuncts and how conjunction and disjunctions evaluated (hint: one way of doing it makes much more sense!). We also find out how to test whether two expressions are observationally equivalent. Finally, we learn about strict and non-strict functions.

Throughout the code, I use the following unparsers for various OCaml types, adapted from Professor Olivier Danvy's lecture notes:

```
let show_bool b =
  (* show_bool : bool -> string *)
  if b
  then "true"
  else "false";;

let show_char c =
  (* show_char : char -> string *)
  "" ^ (if c = '\\' then "\\\" else if c = '\'' then "\\'" else String.make 1
c) ^ "";;

let show_string s =
  (* show_string : string -> string *)
  "\"" ^ s ^ "\"";;

let show_int n =
  (* show_int : int -> string *)
  if n < 0
  then "(" ^ string_of_int n ^ ")"
  else string_of_int n;;

let show_unit () =
  (* show_unit : unit -> string *)
  "()";;

(* ***** *)

let an_int n =
  (* an_int : int -> int *)
```

```

let () = Printf.printf "processing %s...\n" (show_int n)
in n;;

let an_int_v1 n =
  (* an_int_v1 : int -> int *)
  let () = Printf.printf "processing v1 %s...\n" (show_int n)
  in n;;

let a_bool b =
  (* a_bool : bool -> bool *)
  let () = Printf.printf "processing %s...\n" (show_bool b)
  in b;;

let a_bool_v1 b =
  (* a_bool_v1 : bool -> bool *)
  let () = Printf.printf "processing v1 %s...\n" (show_bool b)
  in b;;

let a_bool_v2 b =
  (* a_bool_v2 : bool -> bool *)
  let () = Printf.printf "processing v2 %s...\n" (show_bool b)
  in b;;

let a_char c =
  (* a_char : char -> char *)
  let () = Printf.printf "processing %s...\n" (show_char c)
  in c;;

let a_string s =
  (* a_string : string -> string *)
  let () = Printf.printf "processing %s...\n" (show_string s)
  in s;;

let a_unit () =
  (* a_unit : unit -> unit *)
  let () = Printf.printf "processing the unit value...\n"
  in ();;

let a_function f =
  (* a_function : ('a -> 'b) -> 'a -> 'b *)
  let () = Printf.printf "processing a function...\n"
  in fun x -> f x;;

```

3.2. question 02

Q. In a function application, are the two sub-expressions (namely the one in position of a function, on the left, and the one that is the actual parameter of the function, i.e., the argument, on the right) evaluated from left to right or from right to left?

Let's look at the output of the following snippet of code:

```
(a_function succ) (an_int 10);;
```

Output:

```

processing 10...
processing a function...
- : int = 11

```

Hence, we can see that the argument is evaluated first, and then the function.

Let's try a function that accepts 2 arguments, perhaps the + function?


```
(a_function (+)) (an_int 3) (an_int 5);;
```

Output:

```
processing 5...
processing 3...
processing a function...
- : int = 8
```

Additionally, arguments are evaluated from right to left!

This fits in perfectly with question 1, where:

1. the integer on the right side of the + operand is evaluated first
2. the integer on the left side of the + operand is evaluated next
3. finally the + operand (or, rather, the "+" function) is evaluated in the end

Finally, maybe also a function that accepts 3 arguments?

```
let mul_3 x y z = x * y * z;;
(a_function (mul_3)) (an_int 2) (an_int 6) (an_int 3);;
```

Output (of 2nd line of code):

```
processing 3...
processing 6...
processing 2...
processing a function...
- : int = 36
```

Here too, the arguments are evaluated from right to left!

Let's investigate the type of the `mul_3` function!

```
# mul_3;;
- : int -> int -> int -> int = <fun>
```

As we can see, it is not a function that accepts three arguments.

Rather, a function accepting multiple arguments is actually a euphemism for what is truly happening underneath the hood. We discovered that in a function application, the argument is evaluated first, and then the function itself. And... what a "function with multiple arguments" actually is, is simply a function that is applied a function that is applied to a function that is applied to an integer. And hence, our rule of function application remains consistent, even if we have "multiple arguments".

this also makes sense, because we cannot apply a function to a value without knowing what that value is! (unless, of course, it is a non-strict function; however, even in that case, as we'll see later

the argument is evaluated first, just because we cannot know whether a function is non-strict or not in advance)

3.3. question 03

Q. When a tuple is constructed, in which order were its components evaluated?

Let's look at a few examples first.

```
let pair_int_int = ((an_int 5), (an_int 2));;
```

```
processing 2...
processing 5...
val pair_int_int : int * int = (5, 2)
```

```
let pair_int_bool = ((an_int 3), (a_bool true));;
```

```
processing true...
processing 3...
val pair_int_bool : int * bool = (3, true)
```

```
let pair_bool_char = ((a_bool false), (a_char 'a'));;
```

```
processing 'a'...
processing false...
val pair_bool_char : bool * char = (false, 'a')
```

Hence, we see that pairs are processed from right to left.

What about a pair of pairs? a pair pair!

```
let pair_pair = (((an_int 3), (a_bool true)), ((a_bool false), (a_char 'a')));;
```

```
processing 'a'...
processing false...
processing true...
processing 3...
val pair_pair : (int * bool) * (bool * char) = ((3, true), (false, 'a'))
```

A pair of pairs is consistent with our earlier processing direction! Its components, i.e. pairs themselves, are evaluated from right to left; subsequently, the components within these pairs are themselves evaluated from right to left!

How about a triple? a quadruple? a quintuple even?

```
let triple = ((an_int 3), (a_bool true), (a_char 'c'));;
```

```
processing 'c'...
processing true...
processing 3...
val triple : int * bool * char = (3, true, 'c')
```

```
let quadruple = ((an_int 3), (a_bool true), (a_bool false), (a_char 'a'));;
```

```
processing 'a'...
processing false...
processing true...
```

```
processing 3...
val quadruple : int * bool * bool * char = (3, true, false, 'a')
```

```
let quintuple = ((an_int 3), (a_bool true), (a_bool false), (a_char 'a'),
(a_string "ida"));;
```

```
processing "ida"...
processing 'a'...
processing false...
processing true...
processing 3...
val quintuple : int * bool * bool * char * string =
(3, true, false, 'a', "ida")
```

Triples, quadruples, and quintuples are also consistent! Their components are evaluated from right to left.

Hence, tuples in general are evaluated from right to left, regardless of the types of their components, of course!

3.4. question 04

Q. Are the following functions equivalent?:

```
(fun x1 -> fun x2 -> e0) e1 e2
(fun (x1, x2) -> e0) (e1, e2)
```

The curried function (the haskell kind, not the indian culinary delight):

```
to evaluate "(fun x1 -> fun x2 -> e0) e1 e2":
application associates to the left, hence the above is equivalent to ((fun x1 ->
fun x2 -> e0) e1) e2
to evaluate "((fun x1 -> fun x2 -> e0) e1) e2", we must first evaluate e2 (from
what we learnt in question 02)
- if evaluating e2 does not terminate then the entire evaluation does not
terminate
- if evaluating e2 raises an error, then the entire evaluation raises this
error
- if evaluating e2 evaluates to v2 with type t2, then
- we evaluate ((fun x1 -> fun x2 -> e0) e1)
- to evaluate ((fun x1 -> fun x2 -> e0) e1), we must first evaluate e1:
- if evaluating e1 does not terminate then the entire evaluation does not
terminate
- if evaluating e1 raises an error, then the entire evaluation raises this
error
- if evaluating e1 evaluates to v1 with type t1, then
- evaluating ((fun x1 -> fun x2 -> e0) e1) reduces to evaluating (fun x2
-> e0) in an environment where x1 denotes v1
- evaluating (fun x2 -> e0) returns a function, then
- evaluating ((fun x1 -> fun x2 -> e0) e1) e2 reduces to applying
"this" function (the function we got from the previous line) to v2
- applying "this" function reduces to evaluating e0 in an environment
where x2 denotes v2.
- because "this" function was evaluated in an environment where x1
denoted v1, the environment of e0 also contains x1 denoting v1.
```

The uncurried function:

```

to evaluate "(fun (x1, x2) -> e0) (e1, e2)", we must first evaluate (e1, e2)
- to evaluate (e1, e2), we first evaluate e2 (from what we learnt in question
03)
  - if evaluating e2 does not terminate then the entire evaluation does not
  terminate
  - if evaluating e2 raises an error, then the entire evaluation raises this
  error
  - if evaluating e2 evaluates to v2 with type t2, then we evaluate e1
    - if evaluating e1 does not terminate then the entire evaluation does not
    terminate
    - if evaluating e1 raises an error, then the entire evaluation raises this
    error
    - if evaluating e1 evaluates to v1 with type t1, then
      - the result of (e1, e2) is the pair (v1, v2), then
        - we evaluate (fun (x1, x2) -> e0)
          - evaluating (fun (x1, x2) -> e0) returns a function, then
          - evaluating "(fun (x1, x2) -> e0) (e1, e2)" reduces to applying
          "this" function (the function we got from the previous line) to (v1, v2)
          - applying the function "this" to (v1, v2) reduces to evaluating e0
          in an environment where x1 denotes v1 and x2 denotes v2

```

To sum up, in both cases:

- e2 is the component that is evaluated first, and, hence,
 - if e2 diverges, then in both cases the evaluation does not terminate
 - if e2 raises an error, in both cases the evaluation raises that errors
 - if e2 has a side effect, in both cases e2 makes the same side effect
 - if e2 yields a value v2, the value is the same in both cases
- e1 is the component that is evaluated next, and, hence,
 - if e1 diverges, then in both cases the evaluation does not terminate
 - if e1 raises an error, in both cases the evaluation raises that errors
 - if e1 has a side effect, in both cases e1 makes the same side effect
 - if e1 yields a value v1, the value is the same in both cases
- finally, if e1 and e2 yielded v1 and v2 respectively, Then
 - e0 is evaluated in an environment where x1 denotes v1 and x2 denotes v2

Hence, each of these expressions give rise to the same evaluation/computation.
Therefore, they are equivalent!

And, so we recall the story of `nat_fold_right` and `nat_parafold_right`.

3.4.1. subsidiary questions

S

Our answer is compatible with question 02 and question 03 as:

- the argument is evaluated first, before the function itself is applied to it
 - hence, `e2` was evaluated first in the first expression, and
 - hence, `(e1, e2)` was evaluated first in the second expression
- tuples, as we saw, are evaluated from right to left (hence, `e2` was evaluated before `e1`)

Assuming that `e0` has type `t0`, `x1` has type `t1`, and `x2` has type `t2`, what are the types of `fun x1 -> fun x2 -> e0` and `fun (x1, x2) -> e0`?

The type of "`fun x1 -> fun x2 -> e0`" is "`t1 -> t2 -> t0`", as seen in the following example:

```
# fun x1 -> fun x2 -> (x2, x1);;  
- : 'a -> 'b -> 'b * 'a = <fun>
```

where `'a` is of type `t1`, `'b` is of type `t2` and `'b * 'a` is of type `t0`!

And, the type of "`fun (x1, x2) -> e0`" is "`t1 * t2 -> t0`", as seen in the following example:

```
# fun (x1, x2) -> (x2, x1);;  
- : 'a * 'b -> 'b * 'a = <fun>
```

where `'a` is of type `t1`, `'b` is of type `t2` and `'b * 'a` is of type `t0`!

3.5. question 05

Given two expressions, `e0` and `e1`, are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):

```
(fun x1 -> e0) e1  
let x1 = e1 in e0
```

The first expression:

```
to evaluate "(fun x1 -> e0) e1", we must first evaluate e1 (from what we learnt  
in question 02)  
- if evaluating e1 does not terminate then the entire evaluation does not  
terminate  
- if evaluating e1 raises an error, then the entire evaluation raises this
```

```

error
- if evaluating e1 evaluates to v1 with type t1, then we evaluate (fun x1 ->
e0)
  - evaluating (fun x1 -> e0) returns a function, then
  - evaluating "(fun x1 -> e0) e1" reduces to applying "this" function (the
function we got from the previous line) to v1
    - applying the function "this" to v1 reduces to evaluating e0 in an
environment where x1 denotes v1

```

Let's move on to the second expression (pun intended):

First, let's figure out what is evaluated first in a `let in` expression:

```

(* let x1 = e1 in e0 *)
let x1 = (an_int_v1 8)
  in (an_int x1) + (an_int 1);;

```

```

processing v1 8...
processing 1...
processing 8...
- : int = 9

```

As we can see, the definition is evaluated first; hence we have the computation below:

```

to evaluate "let x1 = e1 in e0", we must first evaluate e1 (from what we learnt
with the previous output)
- if evaluating e1 does not terminate then the entire evaluation does not
terminate
- if evaluating e1 raises an error, then the entire evaluation raises this
error
- if evaluating e1 evaluates to v1 with type t1, then
  - evaluating "let x1 = e1 in e0" reduces to evaluating e0 in an environment
where x1 denotes v1

```

To sum up, in both cases:

- `e1` is the component that is evaluated first, and, hence,
 - if `e1` diverges, then in both cases the evaluation does not terminate
 - if `e1` raises an error, in both cases the evaluation raises that error
 - if `e1` has a side effect, in both cases `e1` makes the same side effect
 - if `e1` yields a value `v1`, the value is the same in both cases, and then
 - `e0` is evaluated in an environment where `x1` denotes `v1`

Hence, each of these expressions give rise to the same evaluation/computation. Therefore, they are equivalent!

This is regardless of whether `e0`, `e1` or `e2` are pure or impure!

3.6. exercise 06

Q. In a (local or global) let-expression declaring several bindings at once, i.e., with and, are the definienses evaluated from left to right or from right to left?

```
let x1 = (an_int 9) and x2 = (an_int 10) and x3 = (an_int 11);;
```

This yields the following output:

```
processing 9...
processing 10...
processing 11...
val x1 : int = 9
val x2 : int = 10
val x3 : int = 11
```

Hence, we see that definienses are evaluated from left to right!

This made me question whether the following would work:

```
let x1 = 40 and x2 = 2 and x3 = x1 + x2;;
```

This yielded the following error, claiming that x1 was unbound:

```
# let x1 = 40 and x2 = 2 and x3 = x1 + x2;;
Line 1, characters 32-34:
1 | let x1 = 40 and x2 = 2 and x3 = x1 + x2;;
                                ^^
Error: Unbound value x1
```

I realised, that this was because x3 was evaluated in an environment where x1 and x2 were not declared.

3.7. exercise 07

Q. Given three expressions, e0, e1, and e2, are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):

```
let x1 = e1 and x2 = e2 in e0
(fun (x1, x2) -> e0) (e1, e2)
```

```
(* let x1 = e1 and x2 = e2 in e0 *)
let x1 = (an_int 12) and x2 = (an_int 13) in (an_int x1) + (an_int x2);;
```

```
processing 12...
processing 13...
processing 13...
processing 12...
- : int = 25
```

```
(* (fun (x1, x2) -> e0) (e1, e2) *)
(fun (x1, x2) -> (an_int x1) + (an_int x2)) ((an_int 12), (an_int 13));;
```

```
processing 13...
```

```
processing 12...
processing 13...
processing 12...
- : int = 25
```

Hence, the two expressions are not equivalent, as there exists an e_1 , namely "an_{int} 12", and an e_2 , namely "an_{int} 13", which lead to different computations, as we can see by the different outputs.

This is because in the first expression, e_1 is evaluated before e_2 . However, in the second expression, e_2 is evaluated before e_1 . hence, their order of evaluation is different.

3.8. exercise 08

Q. In a Boolean conjunction (&& in infix notation), are the conjuncts (i.e., the arguments of &&, of type bool) evaluated from left to right or from right to left?

Let's use two different versions of `a_bool` to find the true direction in which boolean conjuncts are evaluated.

```
(a_bool true) && (a_bool_v1 true);;
```

```
processing true...
processing v1 true...
- : bool = true
```

```
(a_bool false) && (a_bool_v1 true);;
```

```
processing false...
- : bool = false
```

```
(a_bool true) && (a_bool_v1 false);;
```

```
processing true...
processing v1 false...
- : bool = false
```

```
(a_bool false) && (a_bool_v1 false);;
```

```
processing false...
- : bool = false
```

```
(a_bool true) && (a_bool_v1 true) && (a_bool_v2 true);;
```

Here, we use three different versions of `a_bool` to know the true direction that conjuncts are evaluated. otherwise, it would be equally valid to claim with the output of `(a_bool true) && (a_bool true) && (a_bool true)`, which would be "processing true..." three times that the three conjuncts are evaluated in random order *)

```
processing true...
processing v1 true...
processing v2 true...
- : bool = true
```

```
(a_bool true) && (a_bool_v1 true) && (a_bool_v2 false);;
```

```
processing true...
processing v1 true...
processing v2 false...
```



```
- : bool = false
```

```
(a_bool false) && (a_bool_v1 true) && (a_bool_v2 true);;
```

```
# processing false...  
- : bool = false
```

```
(a_bool false) && (a_bool_v1 false) && (a_bool_v2 false);;
```

```
# processing false...  
- : bool = false
```

Hence, the conjuncts are evaluated from left to right. Further, if the result can be known by only evaluating the first conjunct, the remaining conjuncts will not be evaluated. *)

3.8.1.

S

ubsidary questions

- This design makes sense as we don't need to make all the evaluations if can gather the result in advance. For instance, for "false && b1", we can instantly evaluate the result to false after reading simply the first conjunction. This is because, no matter what the value of b1 is (true or false), the expression will evaluate to false.

This is known as short-circuit evaluation!

reference: https://delimited-continuation.github.io/YSC1212/2020-2021_Sem2/week-07_on-simplifying-ocaml-expressions.html#index-3

this means that second conjunction is only evaluated if the first conjunction is not able tell us the answer of the expression.

- this design would also be helpful in two other cases:

1. boolean disjunction (or) (||)

```
(a_bool true) || (a_bool_v1 true);;
```

```
processing true...  
- : bool = true
```

```
(a_bool false) || (a_bool_v1 true);;
```

```
# processing false...  
processing v1 true...  
- : bool = true
```

```
(a_bool true) || (a_bool_v1 false);;
```

```
processing true...  
- : bool = true
```

```
(a_bool false) || (a_bool_v1 false);;
```

```
# processing false...  
processing v1 false...  
- : bool = false
```

On the side, how about we also try a predefined conjunct?

```
let b2 = false;;
let to_be = b2;;
to_be || not to_be;;
```

```
val b2 : bool = false
# val to_be : bool = false
# - : bool = true
```

And now, traced:

```
(a_bool to_be) || not (a_bool_v1 to_be);;
```

```
processing false...
processing v1 false...
- : bool = true
```

A pun is here, that is indeed no question!

As we can see, conjuncts are again evaluated from left to right. Further, similar to earlier, short-circuit evaluation is applied. Hence, if the first conjunct is true, no matter what the next conjunct(s), we know that the expression will evaluate to true. Therefore, the result can be obtained after only simply evaluating the first conjunct.

2. multiplication with 0

We could also potentially use this in multiplication with 0.

For instance, $0 * x$, will evaluate to 0 no matter what the value of x may be.

However, this is not how OCaml will go about evaluating $0 * x$ (and with good reason), as explained in question 04!

A similar evaluation was also implemented in "stringappendv5", where if and only if s_1 and s_2 are not empty strings is the appendment evaluated. hence, in some way, we only make further evaluation if we cannot already obtain the answer by our original evaluation.

3.9. question 09

Reminders:

- for any expressions e_0 and e_1 , evaluating $e_0 e_1$ requires first evaluating e_1 ;
- if evaluating e_1 completes, it yields a value v_1 (let us say of type t_1 , for some t_1), then evaluating $e_0 e_1$ requires evaluating e_0 ;
- if evaluating e_0 completes, it yields a value v_0 , which is a function (let us say of type $t_1 \rightarrow t_2$, for some t_2), due to the typing constraints of OCaml;
- evaluating $e_0 e_1$ then reduces to applying v_0 to v_1 ;
- if this application completes, it yields a value v_2 of type t_2 , due to the typing constraints of OCaml;

- for any expressions e_1 and e_2 , $e_1 * e_2$ is syntactic sugar for `Int.mul e1 e2`, or more precisely for `(Int.mul e1) e2` since application associates to the left;
- So evaluating $e_1 * e_2$ takes the same steps as evaluating `(Int.mul e1) e2`:
- evaluating `(Int.mul e1) e2` requires first evaluating e_2 ;
- if evaluating e_2 completes, it yields a value v_2 , which is of type `int`, due to the typing constraints of OCaml then evaluating `(Int.mul e1) e2` requires evaluating `Int.mul e1`;
- evaluating `Int.mul e1` requires first evaluating e_1 ;
- if evaluating e_1 completes, it yields a value v_1 , which is of type `int`, due to the typing constraints of OCaml; then evaluating `Int.mul e1` requires evaluating `Int.mul`;
- evaluating `Int.mul` yields the value this variable denotes in the current environment; this value is a function of type `int -> int -> int`, due to the typing constraints of OCaml;
- evaluating `Int.mul e1` then reduces to applying this function to v_1 ; the result is a function of type `int -> int`, due to the typing constraints of OCaml;
- evaluating `(Int.mul e1) e2` then reduces to applying this function to v_2 ; the result is a value of type `int`, due to the typing constraints of OCaml; more precisely, this value is the result of multiplying v_1 by v_2 , because the function denoted by `Int.mul` implements the multiplication function.

Here is a simplified version:

- evaluating $e_1 * e_2$ requires first evaluating e_2 ;
- if evaluating e_2 completes, it yields a value v_2 , which is of type `int`, due to the typing constraints of OCaml; then evaluating $e_1 * e_2$ requires evaluating e_1 ;
- if evaluating e_1 completes, it yields a value v_1 , which is of type `int`, due to the typing constraints of OCaml;
- evaluating $e_1 * e_2$ then reduces to multiplying v_1 by v_2 ; the result is a value of type `int`, due to the typing constraints of OCaml.

3.9.1.

or any pure expression v of type `int`, would it be valid to simplify $v * 0$ into `0`?

F

The case of $v * 0$:

- first of all, we note that the expressions "v" and "0" are pure.
- evaluating "v * 0" requires first evaluating 0. evaluating 0 yields 0. then evaluating "v * 0" requires evaluating v.
- since v is a pure expression, evaluating v, a variable of type int, silently yields its value in the current environment, v1, an Integer.
- then, evaluating "v * 0" reduces to multiplying 0 by v1.
- since 0 is absorbing for multiplication, 0 multiplied by v1 evaluates to 0.

hence, the result of evaluating "v * 0" is 0.

The case of 0:

- evaluating "0" yields 0.
- hence, the result of evaluating "0" is 0.

To sum up, in both cases:

- the evaluation silently yields 0.

Hence, it is valid to simplify $0 * v$ into 0 as they both are observationally equivalent, given that v is pure, of course.

3.9.2.

F

or any potentially impure expression e of type int, would it be valid to simplify $e * 0$ into 0?

The case of $e * 0$:

```
(an_int 14) * 0;;
```

```
processing 14...
- : int = 0
```

The case of 0:

```
0;;
```

```
- : int = 0
```

Hence, it is invalid to simplify $e * 0$ to 0, if e is potentially impure; as there exists an e, namely (an_int 14) such that $e * 0$ and 0 are not observationally equivalent.

3.9.3.

F

or any pure expression v of type int, would it be valid to simplify $v * 1$ into v?

The case of $v * 1$:

- first of all, we note that the expressions "v" and "1" are pure.
- evaluating "v * 1" requires first evaluating 1. evaluating 1 yields 1. then evaluating "v * 1" requires evaluating v.
- since v is a pure expression, evaluating v, a variable of type int, silently yields its value in the current environment, v1, an Integer.
- then, evaluating "v * 1" reduces to multiplying 1 by v1.

- the identity property of multiplication states that any number multiplied by 1 keeps its identity, i.e. the number stays the same. hence, 1 multiplied by v1 evaluates to v1.
hence, the result of evaluating "v * 1" is v1.

The case of v:

- since v is a pure expression, evaluating v, a variable of type int, silently yields its value in the current environment, v1, an Integer.
- hence, the result of evaluating "v" is v1.

Hence, it is valid to simplify $v * 1$ into v as they both are observationally equivalent!

3.9.4.

F

or any potentially impure expression e of type int, would it be valid to simplify $e * 1$ into e?

The case of $e * 1$:

- evaluating "e * 1" requires first evaluating 1. evaluating 1 yields 1. then evaluating "e * 1" requires evaluating e.
- since e may be potentially impure, evaluating e either diverges, or raises an error, or yields a value, possibly performing side effects in passing. this value is an integer, due to the typing constraints of Ocaml;
- if evaluating e does not terminate then the entire evaluation does not terminate
- if evaluating e raises an error, then the entire evaluation raises this error
- if evaluating e evaluates to v1, which is of type int, then;
- evaluating "e * 1" reduces to multiplying 1 by v1.
- the identity property of multiplication states that any number multiplied by 1 keeps its identity, i.e. the number stays the same. hence, 1 multiplied by v1 evaluates to v1.
- hence, the result of evaluating "e * 1" is v1.

The case of e:

- evaluating "e" requires evaluating e
- since e may be potentially impure, evaluating e either diverges, or raises an error, or yields a value, possibly performing side effects in passing. this value is an integer, due to the typing constraints of Ocaml;
- if evaluating e does not terminate (i.e. diverges), then the entire evaluation does not terminate
- if evaluating e raises an error, then the entire evaluation raises this error
- if evaluating e evaluates to v1, which is of type int, then;
- the result of evaluating "e" is v1.

To sum up, in both cases:

- if evaluating e diverges, then evaluating each of the expressions diverges too, which leads us to conclude that in this case, they are observationally equivalent.
- if evaluating e raises an error, then evaluating each of the expressions raises this error, which leads us to conclude that in this case, they are observationally equivalent.
- if evaluating e yields a value, performing side effects in passing, then since e is first

evaluated when evaluating "e * 1", and "e", these side effects occur in the same order. the rest of the evaluations takes place as just described in c., which leads us to conclude that in this case, they are observationally equivalent.

- if evaluating e yields a value, performing no side effects in passing, then e is pure, which as just described in c., leads us to conclude that in this case, they are observationally equivalent. since e may be potentially impure,

Hence, it is valid to simplify $e * 1$ to v, if e is potentially impure

3.10. *exercise 10*

e

3.10.1.

a

. Are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):

```
let x1 = e1 and x2 = e2 in (x1, x2);;
let x2 = e2 and x1 = e1 in (x1, x2);;
```

The first case:

```
let x1 = (an_int 17) and x2 = (an_int 18) in (x1, x2);;
```

```
processing 17...
processing 18...
- : int * int = (17, 18)
```

The second case:

```
let x2 = (an_int 18) and x1 = (an_int 17) in (x1, x2);;
```

```
processing 18...
processing 17...
- : int * int = (17, 18)
```

Hence, the two expressions are not equivalent, as thereee exists an e1, namely "an_int 17", and and e2, namely "an_int 18", which lead to different computations, as we can see by the different outputs.

This is because in the first expression, e1 is evaluated before e2. However, in the second expression, e2 is evaluated before e1. Hence, their order of evaluation is different.

3.10.2.

b

. Are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):

```
let x1 = e1 in let x2 = e2 in (x1, x2);;
let x2 = e2 in let x1 = e1 in (x1, x2);;
```

The first case:

```
let x1 = (an_int 17) in let x2 = (an_int 18) in (x1, x2);;
```

```
processing 17...
processing 18...
- : int * int = (17, 18)
```

The second case:

```
let x2 = (an_int 18) in let x1 = (an_int 17) in (x1, x2);;
```

```
processing 18...
processing 17...
- : int * int = (17, 18)
```

Hence, the two expressions are not equivalent, as there exists an e_1 , namely "an_{int} 17", and an e_2 , namely "an_{int} 18", which lead to different computations, as we can see by the different outputs.

This is because in the first expression, e_1 is evaluated before e_2 . However, in the second expression, e_2 is evaluated before e_1 . Hence, their order of evaluation is different.

3.11. *exercise 11*

e

3.11.1.

a

. Are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):

```
let x1 = v1 and x2 = v2 in (x1, x2);;
let x2 = v2 and x1 = v1 in (x1, x2);;
```

The case of `let x1 = v1 and x2 = v2 in (x1, x2):`

```
- first of all, we note that the expressions "v1" and "v2" are pure.
- to evaluate "let x1 = v1 and x2 = v2 in (x1, x2)", we must first evaluate v1
  (from what we learnt in question 06)
- since v1 is a pure expression, evaluating v1, a variable of type t1, silently
  yields its value in the current environment, v_1 of type t1. then evaluating
  "let x1 = v1 and x2 = v2 in (x1, x2)" requires evaluating v2
- since v2 is a pure expression, evaluating v2, a variable of type t2, silently
  yields its value in the current environment, v_2 of type t2.
- then, evaluating "let x1 = v1 and x2 = v2 in (x1, x2)" reduces to evaluating
  (x1, x2) in an environment, G, where x1 denotes v_1 of type t1 and x2 denotes
  v_2 of type t2
  - to evaluate (x1, x2) in G, we first evaluate x2 (from what we learnt in
    question 03)
    - evaluating x2 silently yields its value in the current environment, v_2 of
      type t2, then we evaluate x1
    - evaluating x1 silently yields its value in the current environment, v_1 of
      type t1, then
    - evaluating (x1, x2) in G yields (v1, v2), which is the result of the entire
      evaluation
```

The case of `let x2 = v2 and x1 = v1 in (x1, x2);;`

```
- first of all, we note that the expressions "v1" and "v2" are pure.
- to evaluate "let x2 = v2 and x1 = v1 in (x1, x2)", we must first evaluate v2
```

(from what we learnt in question 06)

- since v_2 is a pure expression, evaluating v_2 , a variable of type t_2 , silently yields its value in the current environment, v_2 of type t_2 . then evaluating "let $x_2 = v_2$ and $x_1 = v_1$ in (x_1, x_2) " requires evaluating v_1
- since v_1 is a pure expression, evaluating v_1 , a variable of type t_1 , silently yields its value in the current environment, v_1 of type t_1 .
- then, evaluating "let $x_2 = v_2$ let $x_1 = v_1$ in (x_1, x_2) " reduces to evaluating (x_1, x_2) in an environment, G , where x_1 denotes v_1 of type t_1 and x_2 denotes v_2 of type t_2
 - to evaluate (x_1, x_2) in G , we first evaluate x_2 (from what we learnt in question 03)
 - evaluating x_2 silently yields its value in the current environment, v_2 of type t_2 , then we evaluate x_1
 - evaluating x_1 silently yields its value in the current environment, v_1 of type t_1 , then
 - evaluating (x_1, x_2) in G yields (v_1, v_2) , which is the result of the entire evaluation

To sum up, in both cases:

- the evaluation silently yields (v_1, v_2)

Hence, these two expressions are equivalent!

3.11.2.

b

. Are the two following expressions equivalent (i.e., does evaluating them carry out the same computation?):

```
let x1 = v1 in let x2 = v2 in (x1, x2);;
let x2 = v2 in let x1 = v1 in (x1, x2);;
```

The case of $\text{let } x_1 = v_1 \text{ in let } x_2 = v_2 \text{ in } (x_1, x_2)$:

- first of all, we note that the expressions " v_1 " and " v_2 " are pure.
- to evaluate " $\text{let } x_1 = v_1 \text{ in let } x_2 = v_2 \text{ in } (x_1, x_2)$ ", we must first evaluate v_1 (from what we learnt in question 06)
- since v_1 is a pure expression, evaluating v_1 , a variable of type t_1 , silently yields its value in the current environment (the current environment being G_0), v_1 of type t_1 . then evaluating " $\text{let } x_1 = v_1 \text{ in let } x_2 = v_2 \text{ in } (x_1, x_2)$ " reduces to evaluating " $\text{let } x_2 = v_2 \text{ in } (x_1, x_2)$ " in an environment, G_1 , where x_1 denotes v_1 of type t_1
- to evaluate " $\text{let } x_2 = v_2 \text{ in } (x_1, x_2)$ ", we must first evaluate v_2
- since v_2 is a pure expression, evaluating v_2 , a variable of type t_2 , silently yields its value in the current environment, v_2 of type t_2 .
- then, evaluating " $\text{let } x_2 = v_2 \text{ in } (x_1, x_2)$ " reduces to evaluating (x_1, x_2) in an environment, G_2 , where x_2 denotes v_2 of type t_2 and x_1 denotes v_1 of type t_1
 - to evaluate (x_1, x_2) in G_2 , we first evaluate x_2 (from what we learnt in question 03)
 - evaluating x_2 silently yields its value in the current environment, v_2 of type t_2 , then we evaluate x_1
 - evaluating x_1 silently yields its value in the current environment, v_1 of type t_1 , then
 - evaluating (x_1, x_2) in G_2 yields (v_1, v_2) , which is the result of the entire evaluation

The case of $\text{let } x_2 = v_2 \text{ in let } x_1 = v_1 \text{ in } (x_1, x_2)$:

- first of all, we note that the expressions "v1" and "v2" are pure.
- to evaluate "let x2 = v2 in let x1 = v1 in (x1, x2)", we must first evaluate v2 (from what we learnt in question 06)
- since v2 is a pure expression, evaluating v2, a variable of type t2, silently yields its value in the current environment (the current environment being G0), v_2 of type t2. then evaluating "let x2 = v2 in let x1 = v1 in (x1, x2)" reduces to evaluating "let x1 = v1 in (x1, x2)" in an environment, G1, where x2 denotes v_2 of type t2
- to evaluate "let x1 = v1 in (x1, x2)", we must first evaluate v1
- since v1 is a pure expression, evaluating v1, a variable of type t1, silently yields its value in the current environment, v_1 of type t1.
- then, evaluating "let x1 = v1 in (x1, x2)" reduces to evaluating (x1, x2) in an environment, G2, where x1 denotes v_1 of type t1 and x2 denotes v_2 of type t2
 - to evaluate (x1, x2) in G2, we first evaluate x2 (from what we learnt in question 03)
 - evaluating x2 silently yields its value in the current environment, v_2 of type t2, then we evaluate x1
 - evaluating x1 silently yields its value in the current environment, v_1 of type t1, then
 - evaluating (x1, x2) in G2 yields (v1, v2), which is the result of the entire evaluation

To sum up,

In the first case:

- v1 is evaluated in the environment G0
- v2 is evaluated in the environment G1

In the second case:

- v1 is evaluated in the environment G1
- v2 is evaluated in the environment G0

Hence, we see how v1 and v2 may yield different values in the two expressions.
Therefore, the two expressions are not observationally equivalent.

To wit:

```
let x1 = 5;;
let x2 = 10;;
```

```
# val x1 : int = 5
# val x2 : int = 10
```

The first case:

```
let x1 = x1 + x2 in let x2 = x1 + x2 in (x1, x2);;
```

```
# - : int * int = (15, 25)
```

The second case:

```
let x2 = x1 + x2 in let x1 = x1 + x2 in (x1, x2);;
```

```
# - : int * int = (20, 15)
```

However, if v1 and v2 do not contain either, x1 or x2, then they become observationally equivalent.

This is because G1 only adds either x1 or x2 to the current environment. Hence, if either of these variables that have been newly added to the environment are not used in v1 or v2, then evaluating them in G1 would be the same as evaluating them in G0.

Therefore, if v1 and v2 do not contain either, x1 or x2, then they become observationally equivalent. (to clarify: they must not contain x1 and they must not contain x2)

3.12. *exercise 12*

e

```
let fun_non_strict _ = 42;; (* assigning to the function the meaning of life *)
fun_non_strict (Random.int 100);; (* applying the function to a *)
```

```
- : int = 42
```

```
fun_non_strict ("any argument, no really, any!");; (* applying the function to a
string *)
```

```
- : int = 42
```

```
fun_non_strict ('a');; (* applying the function to a char *)
```

```
- : int = 42
```

```
fun_non_strict (false);; (* applying the function to a bool *)
```

```
- : int = 42
```

```
fun_non_strict ();; (* applying the function to the unit value *)
```

```
- : int = 42
```

```
fun_non_strict (fun x -> x + 1);; (* applying the function to a function *)
```

```
- : int = 42
```

I mean, since we're not really using the argument, is the function still only evaluated after the argument?

```
(a_function fun_non_strict) (an_int (Random.int 100));;
```

```
processing 30...
processing a function...
- : int = 42
```

```
(a_function fun_non_strict) (a_string "any argument, no really, any!");;
```

```
processing "any argument, no really, any!"...
processing a function...
- : int = 42
```

```
(a_function fun_non_strict) (a_char 'a');;
```

```
processing 'a'...  
processing a function...  
- : int = 42
```

```
(a_function fun_non_strict) (a_bool false);;
```

```
processing false...  
processing a function...  
- : int = 42
```

Yes! Hence, what we observe is consistent with question 2!

We don't really know in advance whether the function we have to "apply" to our argument uses the argument or not!

Hence, it is the case in OCaml as well that applying a non-strict function to any argument yields the same computation!

P.S. but not necessarily the same result, for example consider the function:

```
let funnonstrict _ = Random.int 100;;
```

This function may yield different results every time it is applied to different arguments; however, it will carry out the same computation, i.e. yielding a random integer between 0 and 99 (inclusive)!

Reference:

https://delimited-continuation.github.io/YSC1212/2020-2021_Sem2/week-01_recap.html#a-plea-for-precision

3.13.

onclusion

c

And, now we can say that we know OCaml inside-out and outside-in! We learnt about the rationale of OCaml's creation, by which we can further think about how to create our very own version of OCaml, and what decisions we would have to take to do that. We also saw how creating differet versions of unparsers helped us see the order of evaluation of OCaml. We learnt about when two expressions may be equivalent and when two expressions may not be equivalent! Onward to strings!

4. strings

4.1. introduction

Strings! You're reading one right now! Not the string-theory kind or the thread kind, but strings made out of characters. We start this mini-project by creating our concatenation function. Next, we implement our very own version of `String.map` and also learn how to concatenate chars!

Subsequently, we create our very own version of `String.mapi`, which, of course, is much easier to create now that we have already created `String.map`. While creating `String.map` and `String.mapi`, we also take what we learnt from the underlying determinism of OCaml and apply it; now, we become the creators, and tell OCaml what direction to run the evaluation in! We implement string reversal, and here too we see that we are able to decide what direction we want our evaluation to run in. We study the relations between string concatenation and string reversal. Next, we implement a palindrome generator and palindrome detectors. We also reverse a palindrome, and then reverse it

again, and reverse it one last time! Finally, we define `String.map` using `String.map` and `String.map_i` with the help of `String.init`.

I use the following definition of `nat_parafold_right`, adapted from Professor Olivier Danvy's lecture notes, throughout the rest of my code.

```
(* nat_parafold_right *)
let nat_parafold_right zero_case succ_case n_given =
  (* nat_parafold_right : 'a -> (int -> 'a -> 'a) -> int -> 'a *)
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = 0
    then zero_case
    else let n' = n - 1
         in let ih = visit n'
         in succ_case n' ih      (* <-- succ_case takes two arguments *)
  in visit n_given;;
```

4.2. question 01

Q. Implement an OCaml function of type `string -> string -> string` that concatenates two strings, using `String.init`.

Over here, we're essentially creating our version of the inbuilt `^` function.

Let's first take a look at what `String.init` does:

```
# String.init;;
- : int -> (int -> char) -> string = <fun>
```

`String.init`, as the name suggests, initialises a string.

It takes an integer (the length of the string we want to initialise), and a function that maps an integer to a character. This function maps every index to a certain character.

And, so we have one building block with us: `String.init`.

Wait a second, we're forgetting something else: a unit test! So let's start with that:

```
(* a unit test for our future string_append function *)
let test_string_append candidate =
  (* empty string: *)
  let b0 = (candidate "" "" = "")
  (* some intuitive cases: *)
  and b1 = (candidate "abc" "def" = "abcdef")
  and b2 = (candidate "AbC" "dEf" = "AbCdEf")
  and b3 = (candidate "hello, " "world!" = "hello, world!")
  and b4 = (candidate "\\\" \"\" = "\\\" \"")
  and b5 = (candidate " " " " = " ")
  (* random case *)
  and b6 = (let a = String.init 10 (fun _ -> char_of_int (Random.int 128))
            and b = String.init 10 (fun _ -> char_of_int (Random.int 128))
```

```

        in candidate a b = a ^ b)
(* etc. *)
in b0 && b1 && b2 && b3 && b4 && b5 && b6;;

```

And, as we can see, we use `String.init` in our unit test as well (though, the function ignores the indices and just creates random characters).

We also use `^` as a witness function to conduct a positive random test.

Let's now think about creating our string append function.

Well, we would want to initialise a string that is equal to the sum of the lengths of the two input strings. Further, the function that we pass to `String.init` should be able to differentiate when we should append a character from the first string, and when we should append a character from the second string; aaanddd this fits the bill!:

```

let string_append_v1 s1 s2 =
  String.init (String.length s1 + String.length s2)
  (fun i -> if i < (String.length s1)
            then String.get s1 i
            else String.get s2 (i - (String.length s1)));;

(* testing function with unit test *)
let () = assert (test_string_append string_append_v1);;

```

And, of course, the function passes our unit test.

BUT, it seems quite inefficient honestly. Let's trace `String.length` and see how many times it is called by using a traced version of `String.length`:

```

(* traced version of String.length *)
let string_length s1 =
  let () = Printf.printf "string length %s -> \n" s1
  in String.length s1;;

let string_append_traced_v1 s1 s2 =
  String.init (string_length s1 + string_length s2)
  (fun i -> if i < (string_length s1)
            then String.get s1 i
            else String.get s2 (i - (string_length s1)));;

```

```

# string_append_traced_v1 "abc" "def";;
string length def ->
string length abc ->
string length abc ->
string length abc ->
string length abc ->
string length abc ->
string length abc ->
string length abc ->
string length abc ->
string length abc ->
string length abc ->
- : string = "abcdef"

```

As we can see, in this version, `String.length` is called many times on `s1`, and once on `s2`!

We can improve this function to only call `String.length` twice, in total!

```

let string_append_v2 s1 s2 =
  let s1_l = String.length s1
  and s2_l = String.length s2 in
  String.init
    (s1_l + s2_l)
  (fun i -> if i < s1_l
             then String.get s1 i
             else String.get s2 (i - s1_l));;

let string_append_traced_v2 s1 s2 =
  let s1_l = string_length s1
  and s2_l = string_length s2 in
  String.init
    (s1_l + s2_l)
  (fun i -> if i < s1_l
             then String.get s1 i
             else String.get s2 (i - s1_l));;

(* testing function with unit test *)
let () = assert (test_string_append string_append_v2);;

```

And, of course, the function passes our unit test.

And, as we can see, we only call `String.length` twice now: once on `s1`, once on `s2`!

```

# string_append_traced_v2 "abc" "def";;
string length abc ->
string length def ->
- : string = "abcdef"

```

Ahh much better!

BUT wait, there's still scope for improvement!

Well, if either `s1` or `s2` are empty strings, we don't need to call `String.init` at all; if we know a string is empty, we can simply return the other string! And so we have `v3`:

```

let string_append_v3 s1 s2 =
  let s1_l = String.length s1
  and s2_l = String.length s2 in
  if s1_l = 0
  then s2
  else if s2_l = 0
  then s1
  else
    String.init
      (s1_l + s2_l)
    (fun i -> if i < s1_l
               then String.get s1 i
               else String.get s2 (i - s1_l));;

(* testing function with unit test *)
let () = assert (test_string_append string_append_v3);;

```

And, of course, the function passes our unit test.

Ahh, ever so slightly better. Our function will now use less space in the case that one of the strings is empty.

BUT wait a second, if we know that `s1` is an empty string, why calculate the length of `s2` at all? And so we have `v4`:

```
let string_append_v4 s1 s2 =
  let s1_l = String.length s1 in
  if s1_l = 0
  then s2
  else let s2_l = String.length s2 in
        if s2_l = 0
        then s1
        else
          String.init
            (s1_l + s2_l)
            (fun i -> if i < s1_l
                      then String.get s1 i
                      else String.get s2 (i - s1_l));;

(* testing function with unit test *)
let () = assert (test_string_append string_append_v4);;
```

And, of course, the function passes our unit test.

Ahh, ever so slightly better. Our function will now not call `String.length` unnecessarily if it is not needed! (this feels similar to short-circuit evaluation!)

BUT wait a second, well, if `s1` or `s2` are empty, do we really need to call `String.length` at all? We can simply compare them to an empty string! And so we have `v5`

```
let string_append_v5 s1 s2 =
  if s1 = ""
  then s2
  else if s2 = ""
  then s1
  else let s1_l = String.length s1
        and s2_l = String.length s2 in
        String.init
          (s1_l + s2_l)
          (fun i -> if i < s1_l
                    then String.get s1 i
                    else String.get s2 (i - s1_l));;

(* testing function with unit test *)
let () = assert (test_string_append string_append_v5);;
```

And, of course, the function passes our unit test. And so we have it; the road to wisdom! Incremental improvements, slowly but steadily!

4.3. question 09

Q.Implement an OCaml function that given 3 characters, yields a string containing these 3 characters.

This implementation should pass the following unit test:

```
let test_warmup candidate =  
  (candidate 'a' 'b' 'c' = "abc");;
```

String.make may be useful here! Let's take a look at what it does:

```
# String.make;;  
- : int -> char -> string = <fun>
```

As the name suggests, String.make *makes* a string of a particular length consisting of only a single character repeated multiples times!

The function takes an int, which will be the length of the string we want to make. It also takes a char that will be repeated throughout the string.

The reason we need String.make at all is because we cannot concatenate chars; we can only concatenate strings! Hence, we would have to convert our chars to strings before concatenating them, like so:

```
let warmup_v0 c0 c1 c2 =  
  let c0_string = String.make 1 (c0)  
  and c1_string = String.make 1 (c1)  
  and c2_string = String.make 1 (c2)  
  in string_append_v4 (string_append_v4 c0_string c1_string) c2_string;;  
  
(* testing function with unit test *)  
let () = assert (test_warmup warmup_v0);;
```

And, of course, the function passes our unit test.

BUT wait, there's something else we can do! Everytime we run String.make, we create a new string; however, we can use String.init so that we only create one string!

The function we'll pass to String.init will return c0 if on index 0, c1, if on index 1, and finally, if neither on index 0 or 1, it'll return c3. BEHOLD:

```
let warmup_v1 c0 c1 c2 =  
  String.init 3 (fun i -> if i = 0  
                           then c0  
                           else if i = 1  
                           then c1  
                           else c2);;  
  
(* testing function with unit test *)  
let () = assert (test_warmup warmup_v1);;
```

And, of course, the function passes our unit test.

4.4. question 02

Let's explore String.map, and try implementing it ourself as well.

4.4.1.

n which order are the characters accessed in the given string?

We can find out what order the characters are accessed in by creating a traced version of `String.map`!

BUT first, we'll need an unparser for chars, like so:

```
let show_char c =
  (* show_char : char -> string *)
  "" ^ (if c = '\\' then "\\\" else if c = '\'' then "\\'" else String.make 1
c) ^ "";;

let a_char c =
  (* a_char : char -> char *)
  let () = Printf.printf "processing %s...\n" (show_char c)
  in c;;
```

PERFECT! Now, onto creating a traced version of `String.map`. We don't really need a new version at all; instead, we can simply pass the function `fun c -> (a_char c)` to `String.map`! Like so:

```
String.map (fun c -> (a_char c)) "Yal'";;
```

And let's see what output we get:

```
index 0
processing 'Y'...
index 1
processing 'a'...
index 2
processing 'l'...
index 3
processing '\''...
- : string = "Yal'"
```

Hence, we see that characters are accessed from left to right!

4.4.2.

sing a recursive function that operates over a natural number, implement a left-to-right version of `String.map`. Your implementation will therefore simulate a for-loop that goes from the smallest index to the largest index in the string.

Let's think about how we may create our own left-to-right version of `String.map`

Let's first set up a function to test our own `string_map` function:

```
(* a function to test our new string_map_up function *)
let test_string_map_up str_map_up f s =
  (str_map_up f s = String.map f s);;
```

Let's first try out a simple recursive implementation and think about whether that would map up or map down, and why!

For our recursive implementation, we can simply keep on running the given function on a particular character and concatenating it with the remainder of the string (that would be represented by `ih`)! Behold:

```
let string_map_up_v0 f s =
  let s_length = String.length s in
  let rec visit n =
    if n = 0
    then String.make 1 (f (String.get s 0))
    else let n' = n - 1 in
         let ih = visit n' in
         ih ^ (String.make 1 (f (String.get s n))) in
  visit (s_length - 1);;

(* testing function with testing function *)
let () = assert (test_string_map_up
  string_map_up_v0
  (fun c -> char_of_int (int_of_char c + 1))
  "abc");;
```

And, of course, the function passes our unit test.

However, let's see what direction it traces in?

```
# string_map_up_v0 (fun c -> a_char c) "abc";;
processing 'a'...
processing 'b'...
processing 'c'...
- : string = "abc"
```

If we trace it, we can see it accesses characters upwards!

If we really think about it, this is consistent with what we learnt in question 05 of the project about the underlying determinism of ocaml!

In the expression `let x1 = e1 in e0`, the definiens `let x1 = e1` is evaluated first. then, the expression `e0` is evaluated!

Hence, in the following code snippet:

```
let ih = visit n' in
  ih ^ (String.make 1 (f (String.get s n))) in
visit (s_length - 1);;
```

We first evaluate `let x1 = e1`, that is `let ih = visit n'`.

To evaluate `let ih = visit n'`, we must first evaluate `visit n'`.

Hence, when we evaluate `visit (s_length - 1)`, we run the same expression as above, let `ih = visit n'`. for the expression above, we must evaluate `visit n'` in an environment where `n'` denotes `n - 1`, or in this case `s_length - 2`.

This chain continues all the way down, until the very last character, that is `visit 0`. Hence, `visit 0` is what is evaluated before anything else!

`visit 0` evaluates the first character of the given string. It passes this value to `visit 1`, which subsequently evaluates the second character of the string and so on until we reach `visit (s_length - 1)`.

Hence, the function first accesses the character at the 0th indice, then the 1st indice, then 2nd and so on until it reaches `s_length - 1`.

And so we see how the functions maps "up"!

Okay, now we know how it maps up, but there's still scope for improvement!

For one, the version we just implemented will not work for empty strings! Hence, let's account for that with the version below:

```
let string_map_up_v1 f s =
  let s_length = String.length s in
  if s_length = 0
  then s
  else let rec visit n =
    if n = 0
    then String.make 1 (f (String.get s 0))
    else let n' = n - 1 in
         let ih = visit n' in
         ih ^ (String.make 1 (f (String.get s n))) in
    visit (s_length - 1);;

(* testing function with testing function *)
let () = assert (test_string_map_up
  string_map_up_v1
  (fun c -> char_of_int (int_of_char c + 1))
  "");;

(* the new function now works fine with empty strings! *)

(* testing function with testing function *)
let () = assert (test_string_map_up
  string_map_up_v1
  (fun c -> char_of_int (int_of_char c + 1))
  "abc");;
```

Of course, the new function also passes our earlier test, and the direction of character access is the same as well!

```
# string_map_up_v1 (fun c -> a_char c) "abc";;  
processing 'a'...  
processing 'b'...  
processing 'c'...  
- : string = "abc"
```

However, having a separate if condition to deal with the base case seems a bit odd!

We should NOT have to define our base case separately

Harumph.

Maybe we need to rethink our function to truly define it inductively.

Base case: $n = 0$

- we must return an empty string!

Inductive step: $n' = n - 1$

- apply given function to character at that index and concatenate!

And so, we can try implementing our `string_map_up` function *based* (pun intended) on this new base case!

```
let string_map_up_v2 f s =  
  let s_length = String.length s in  
  let rec visit n =  
    if n = 0  
    then ""  
    else let n' = n - 1 in  
          let ih = visit n' in  
          ih ^ (String.make 1 (f (String.get s n')) in  
  visit s_length;;  
  
(* testing function with testing function *)  
let () = assert (test_string_map_up  
  string_map_up_v2  
  (fun c -> char_of_int (int_of_char c + 1))  
  "");;  
  
(* testing function with testing function *)  
let () = assert (test_string_map_up  
  string_map_up_v2  
  (fun c -> char_of_int (int_of_char c + 1))  
  "abc");;
```

And, of course, the function passes our unit test.

Eureka!

andddd.. drumroll... now to trace it:

```
# string_map_up_v2 (fun c -> a_char c) "abc";;
processing 'a'...
processing 'b'...
processing 'c'...
- : string = "abc"
```

YAY!

Maybe with `nat_parafold_right` too?

```
let string_map_up_v3 f s =
  nat_parafold_right ""
    (fun n' ih -> ih ^ (String.make 1 (f (String.get s n'))))
    (String.length s);;

(* testing function with testing function *)
let () = assert (test_string_map_up
  string_map_up_v3
  (fun c -> char_of_int (int_of_char c + 1))
  "abc");;

(* testing function with testing function *)
let () = assert (test_string_map_up
  string_map_up_v3
  (fun c -> char_of_int (int_of_char c + 1))
  "");;
```

And, of course, the function passes our unit test.

And, just one last time, to trace it, just to make sure:

```
# string_map_up_v3 (fun c -> a_char c) "abc";;
processing 'a'...
processing 'b'...
processing 'c'...
- : string = "abc"
```

And hence, we were able to define `string_map_up` inductively!

Onward to `mapdown`!

4.4.3.

U

sing a recursive function that operates over a natural number, implement a right-to-left version of `String.map`. Your implementation will therefore simulate a for-loop that goes from the largest index to the smallest index in the string.

Let's now think about how we may create our own right-to-left version of `String.map`

Let's first set up a function to test our own `stringmap` function:

```
(* a function to test our potential string_map_down function *)
let test_string_map_down str map_down f s =
```

```
(str_map_down f s = String.map f s);;
```

Let me let you on in a little secret:

It's really the same function as `teststringmapup`!

:-)

(P.S. it doesn't check for the traces anyways, and it compares it to `String.map`!)

(P.P.S but, please don't tell prof that!)

Okay, so we know a simple recursive definition will not work like last time, as that would just map upwards!

Hmmmm. Okay, the main idea there was that we applied `visit` to the string's length - 1 first! That ways the `let e0 in e1` ran `e0` first, i.e. `visit n'` first, which in turn meant that the same thing happened for each subsequent run of `visit n'` until what was evaluated at the very beginning was `visit 0`!

Sooo, given that, why don't we just flip it? Let's try applying `visit` to 0 to begin with and make the inductive step be to apply `visit` to `n'` BUT where `n'` represents `n + 1`, instead of `n - 1`.

And, so we have:

```
let string_map_down_v0 f s =
  let s_length = String.length s in
  if s_length = 0
  then s
  else let rec visit n =
    if n = s_length - 1
    then String.make 1 (f (String.get s (s_length - 1)))
    else let n' = n + 1 in
         let ih = visit n' in
         (String.make 1 (f (String.get s n))) ^ ih in
    visit 0;;

(* testing function with testing function *)
let () = assert (test_string_map_down
  string_map_down_v0
  (fun c -> char_of_int (int_of_char c + 1))
  "abc");;

(* testing function with testing function *)
let () = assert (test_string_map_down
  string_map_down_v0
  (fun c -> char_of_int (int_of_char c + 1))
  "");;
```

And, of course, the function passes our unit test.

Of course, the new function also works with empty strings, as we used what we learnt from the previous exercise!

And to trace it to see the direction of character access:

```
# string_map_down_v0 (fun c -> a_char c) "abc";;  
processing 'c'...  
processing 'b'...  
processing 'a'...  
- : string = "abc"
```

If we really think about it, this is consistent with what we learnt in question 05 of the project about the underlying determinism of ocaml!

In the expression `let x1 = e1 in e0`, the definiens, `let x1 = e1`, is evaluated first. Then, the expression `e0` is evaluated!

Hence, in the following code snippet:

```
let ih = visit n' in  
  (String.make 1 (f (String.get s n))) ^ ih in  
visit 0;;
```

We first evaluate `let x1 = e1`, that is `let ih = visit n'`.

To evaluate `let ih = visit n'`, we must first evaluate `visit n'`!

Hence, when we evaluate `visit 0` we run the same expression as above, `let ih = visit n'`. for the expression above, we must evaluate `visit n'` in an environment where `n'` denotes `n + 1`, or in this case `1`.

This chain continues all the way down, until the very last character, that is `visit s_length - 1`. Hence, `visit s_length - 1` is what is evaluated before anything else!

`visit s_length - 1` evaluates the last character of the given string. It passes this value to `visit s_length - 2`, which then evaluates the second last character of the string and so on until we reach `visit 0`.

Hence, the function first accesses the character at the indice `s_length - 1`, then the one at indice `s_length - 2`, then `s_length - 3` and so on until it reaches `0`!

And so we see how the functions maps "down"!

WAIT a second.....

"learnt from our previous exercise"

LIES !!!

We are repeating our mistake!

The fact that we are still using a separate if condition for strings of length 0 suggests that our definition is not truly inductively defined. We should NOT have to define our base case separately!

Harumph.

We really haven't learnt, have we?

Maybe we need to rethink our function to truly define it inductively.

base case: $n = \text{length}$

- we must return an empty string!

inductive step: $n = n + 1$

- apply given function to character at that index and concatenate!

And so, we can try implementing our `string_map_down` function *based* (pun intended) on this new base case!

```
let string_map_down_v1 f s =
  let s_length = String.length s in
  let rec visit n =
    if n = s_length
    then ""
    else let n' = n + 1 in
          (String.make 1 (f (String.get s n))) ^ visit n' in
  visit 0;;

(* testing function with testing function *)
let () = assert (test_string_map_down
  string_map_down_v1
  (fun c -> char_of_int (int_of_char c + 1))
  "abc");;

(* testing function with testing function *)
let () = assert (test_string_map_down
  string_map_down_v1
  (fun c -> char_of_int (int_of_char c + 1))
  "");;
```

And to trace it:

```
# string_map_down_v1 (fun c -> a_char c) "abc";;
processing 'c'...
processing 'b'...
processing 'a'...
- : string = "abc"
```

Finally! This function seems satisfactory (for now)!

The reason we could define `string_map_up` with `nat_parafold_right` is because `nat_parafold_right` by virtue of its definition begins at `visit n_given`.

How can we implement `string_map_down` with `nat_parafold_right`? We could try writing our own version of `nat_parafold_right` that traverses downwards rather than upwards of course!

```
(* nat_parafold_right_down *)
let nat_parafold_right_down n_case succ_case n_given =
  (* nat_parafold_right_down : 'a -> (int -> 'a -> 'a) -> int -> 'a *)
  let () = assert (n_given >= 0) in
  let rec visit n =
    if n = n_given
    then n_case
    else let n' = n + 1
         in let ih = visit n'
            in succ_case n' ih
  in visit 0;;
```

We can now implement `string_map_down` using this new definition of `nat_parafold_right_down`!

```
(* with this new definition of nat_parafold_right_down, we can implement
string_map_down *)
let string_map_down_v2 f s =
  nat_parafold_right_down ""
    (fun n' ih -> (String.make 1 (f (String.get s (n' -
1)))) ^ ih)
    (String.length s);;

(* testing function with testing function *)
let () = assert (test_string_map_down
  string_map_down_v2
  (fun c -> char_of_int (int_of_char c + 1))
  "");;

(* testing function with testing function *)
let () = assert (test_string_map_down
  string_map_down_v2
  (fun c -> char_of_int (int_of_char c + 1))
  "abc");;
```

And yes! This passes our earlier unit test too! BUT that was expected...

Aaaannddd... the trace! of course!

```
# string_map_down_v2 (fun c -> a_char c) "abc";;
processing 'c'...
processing 'b'...
processing 'a'...
- : string = "abc"
```

And, there we have it!

4.5. question 03

New function, same story! `String.mapi`, here we come!

4.5.1.

I

n which order are the characters accessed in the given string?

We can find out what order the characters are accessed in by creating a traced version of `String.mapi`!

BUT first, we'll need an unparser for ints, like so:

```
let show_int n =  
  (* show_int : int -> string *)  
  if n < 0  
  then "(" ^ string_of_int n ^ ")"  
  else string_of_int n;;
```

PERFECT! Now, onto creating a traced version of `String.mapi`. We don't really need a new version at all; instead, we can simply pass the function `fun i c -> let () = Printf.printf "index %s\n" (show_int i) in (a_char c)` to `String.mapi`! Like so:

```
String.mapi  
  (fun i c -> let () = Printf.printf "index %s\n" (show_int i) in (a_char c))  
  "Yal";;
```

And, so we have:

```
index 0  
processing 'Y'...  
index 1  
processing 'a'...  
index 2  
processing 'l'...  
index 3  
processing '\n'...  
- : string = "Yal"
```

Hence, we see that characters are accessed from left to right, with 0 as the first character's index and the index being incremented for each additional character!

4.5.2.

U

sing a recursive function that operates over a natural number, implement a left-to-right version of `String.mapi`. Your implementation will therefore simulate a for-loop that goes from the smallest index to the largest index in the string.

Now, we can just use `nat_parafold_right` and `nat_parafold_right_down`, taking a page from what we learnt in our previous exercise!

BUT first, like earlier a function to test whether the function we write is really the same as `String.mapi`:

```
(* a function to test our new string_mapi_up function *)
```

```

let test_string_mapi_up f s =
  (string_mapi_up f s = String.mapi f s);;

let string_mapi_up f s =
  nat_parafold_right ""
    (fun n' ih -> ih ^ String.make 1 (f n' (String.get s
n'))))
    (String.length s);;

(* or, if you would prefer, inlined: *)
(* let string_mapi_up f s =
  let s_length = String.length s in
  let rec visit n =
    if n = 0
    then ""
    else let n' = n - 1 in
         visit n' ^ (String.make 1 (f n' (String.get s n')))) in
  visit s_length;; *)

(* testing function with testing function (and an empty string) *)
let () = assert (test_string_mapi_up
  (fun i c -> char_of_int (int_of_char c + i))
  "");;

(* testing function with testing function *)
let () = assert (test_string_mapi_up
  (fun i c -> char_of_int (int_of_char c + i))
  "abcdef");;

```

And the most important bit: the trace!

```

# string_mapi_up
(fun i c -> let () = Printf.printf "index %s\n" (show_int i) in (a_char c))
"abc";;

index 0
processing 'a'...
index 1
processing 'b'...
index 2
processing 'c'...
- : string = "abc"

```

And it works! And why would it not?

The function iterates over the indices from left to right

Onward to `string_mapidown`

4.5.3.

U

sing a recursive function that operates over a natural number, implement a right-to-left version of `String.mapi`. Your implementation will therefore simulate a for-loop that goes from the largest index to the smallest index in the string.

First, we write our testing function (which again, is really the same as our previous one, but oh well, names are nice!)

```
(* a function to test our new string_mapi_down function *)
let test_string_mapi_down f s =
  (string_mapi_down f s = String.mapi f s) ;;
```

And now, simply using `nat_parafold_right_down` instead:

```
(* onward to string_mapi_down *)
let string_mapi_down f s =
  nat_parafold_right_down ""
    (fun n' ih ->
      String.make 1 (f (n' - 1) (String.get s (n' - 1))) ^
ih)
    (String.length s) ;;

(* or, if you would prefer, inlined: *)
(* let string_mapi_down f s =
  let s_length = String.length s in
  let rec visit n =
    if n = s_length
    then ""
    else let n' = n + 1 in
      (String.make 1 (f n (String.get s n))) ^ visit n' in
  visit 0 ;; *)

(* testing function with testing function (and an empty string) *)
let () = assert (test_string_mapi_down
  (fun i c -> char_of_int (int_of_char c + i))
  "") ;;

(* testing function with testing function *)
let () = assert (test_string_mapi_down
  (fun i c -> char_of_int (int_of_char c + i))
  "abcdef") ;;
```

And the most important bit: the trace!

```
# string_mapi_down
(fun i c -> let () = Printf.printf "index %s\n" (show_int i) in (a_char c))
"abc" ;;

index 2
processing 'c'...
index 1
processing 'b'...
index 0
processing 'a'...
- : string = "abc"
```

And it works! And why would it not?
The function iterates over the indices from right to left!

4.6. question 04

Q. Implement two OCaml functions of type `string -> string` that reverse a string,

Let's first begin by writing a unit test for our `string_reverse` function!

```
(* a unit test for our future string_reverse function *)
let test_string_reverse string_reverse =
  (* empty string: *)
  let b0 = (string_reverse "" = "")
  (* some intuitive cases: *)
  and b1 = (string_reverse "abc" = "cba")
  and b2 = (string_reverse "AbC" = "CbA")
  and b3 = (string_reverse "hello, " = " ,olleh")
  and b4 = (string_reverse "\\\" = "\\\"")
  and b5 = (string_reverse " aba " = " aba ")
  in b0 && b1 && b2 && b3 && b4 && b5;;
```

A-okay! Now, we can go on to writing our own functions!

4.6.1. **ne that uses String.mapi,**

o

Well, for the function with `String.mapi`, we don't want to change anything depending on the character; our main focus will be on the indice!

And, essentially, for every indice `i` we want to get the `s_length - i - 1` character (from the back). We subtract 1 because indices start from 0, and we subtract `i` because as we move ahead in indices from the front, we want to move more backwards from the back! And that's all! So we have:

```
let string_reverse_mapi s =
  String.mapi
    (fun i _ -> let s_length = String.length s in
                  String.get s (s_length - i - 1))
  s;;

(* testing function with unit test *)
let () = assert (test_string_reverse string_reverse_mapi);;
```

And, of course, the function passes our unit test. Wunderbar!

Now, onto an ever so slightly tougher(?) problem!

4.6.2. **nother that uses recursion over a non-negative integer.**

a

Let's think about how we can implement a reversing function with 0. Oh well, for a start, if the length is 0 then we can just return the string!

Further, the aim of `visit n'` would be to get the next character, and we can simply append this character in the "wrong" direction to reverse the string!

And so we have:

```
let string_reverse_rec_v0 s =
  let s_length = String.length s in
  if s_length = 0
  then s
  else
```

```

let rec visit n =
  if n = 0
  then String.make 1 (String.get s 0)
  else let n' = n - 1 in
        (String.make 1 (String.get s n)) ^ visit n' in
  visit (s_length - 1);;

(* testing function with unit test *)
let () = assert (test_string_reverse string_reverse_rec_v0);;

```

It passes the unit test! however;

CONSTANT VIGILANCE! IT SEEMS SO THAT OUR LESSON HAS NOT BEEN LEARNT

We are still using a mechanism to deal with strings of length 0 rather than truly defining our function inductively.

Back to the drawing board! Well, we already implicitly explained our base case and induction step in our previous explanation, so we have:

Base case: $n = 0$

- we must return an empty string!

Inductive step: $n' = n - 1$

- we concatenate the character at that index in the "wrong" direction!

And, so we have:

```

let string_reverse_rec_v1 s =
  let s_length = String.length s in
  let rec visit n =
    if n = 0
    then ""
    else let n' = n - 1 in
          (String.make 1 (String.get s n')) ^ visit n' in
  visit s_length;;

(* testing function with unit test *)
let () = assert (test_string_reverse string_reverse_rec_v1);;

```

And, of course, the function passes our unit test.

This updated version now is truly inductively defined, and, so we can define it using `nat_parafold_right` as well!

Moreso, we can specify what direction we want characters to be accessed by choosing what version of `nat_parafold_right` we want to use!

And, so we have,

Reversing upwards:

```
(* characters accessed from left to right *)
let string_reverse_rec_up_traced s =
  nat_parafold_right ""
    (fun n' ih -> (String.make 1 (a_char (String.get s n')) ^ ih)
      (String.length s));;

(* and now untraced *)
let string_reverse_rec_up s =
  nat_parafold_right ""
    (fun n' ih -> String.make 1 (String.get s n') ^ ih)
      (String.length s);;
```

The trace, of course:

```
# string_reverse_rec_up_traced "abc";;
processing 'a'...
processing 'b'...
processing 'c'...
- : string = "cba"
```

Reversing downwards:

```
(* characters accessed from right to left *)
let string_reverse_rec_down_traced s =
  nat_parafold_right_down ""
    (fun n' ih -> ih ^ String.make 1 (a_char (String.get s
      (pred n'))))
      (String.length s);;

(* and now untraced *)
let string_reverse_rec_down s =
  nat_parafold_right_down ""
    (fun n' ih -> ih ^ String.make 1 (String.get s (pred
      n'))))
      (String.length s);;
```

The trace, of course:

```
# string_reverse_rec_down_traced "abc";;
processing 'c'...
processing 'b'...
processing 'a'...
- : string = "cba"
```

And both implementations, of course, pass our unit test!

```
let () = assert (test_string_reverse string_reverse_rec_up);;
let () = assert (test_string_reverse string_reverse_rec_down);;
```

No Harumph, Hurrah!

4.7. question 05

Q. Implement a unit-test function for `string_reverse` that puts this relation (between `string_reverse s1`, `string_reverse s2`, and `s1 ^ s2`) to the test.

Well, simply said, if we reverse `a ^ b` (the concatenation of `a` and `b`), we get `b` reversed concatenated with `a` reversed! And that is the relation!

So we have:

```
let unit_test_string_reverse_for_question_5 string_reverse =
  (* empty string: *)
  let b0 = (string_reverse "" = "")
  (* some intuitive cases: *)
  and b1 = (string_reverse "abc" = "cba")
  and b2 = (string_reverse "AbC" = "CbA")
  and b3 = (string_reverse "hello, " = " ,olleh")
  and b4 = (string_reverse "\\\\" = "\\")
  and b5 = (string_reverse " aba " = " aba ")
  (* relation between s1, s2 and s1 ^ s2 *)
  and b6 = (let a = String.init 10 (fun _ -> char_of_int (Random.int 128))
             and b = String.init 10 (fun _ -> char_of_int (Random.int 128))
             in (string_reverse b ^ string_reverse a) = string_reverse (a ^ b))
  (* etc. *)
  in b0 && b1 && b2 && b3 && b4 && b5 && b6;;

(* testing function with unit test *)
let () = assert (unit_test_string_reverse_for_question_5 string_reverse_mapi);;
let () = assert (unit_test_string_reverse_for_question_5
string_reverse_rec_v1);;
let () = assert (unit_test_string_reverse_for_question_5
string_reverse_rec_up);;
let () = assert (unit_test_string_reverse_for_question_5
string_reverse_rec_down);;
```

All our previous functions pass our new unit test as well!

4.8. question 06

Q. Implement a generator of palindromes of type `int -> string`, i.e., an OCaml function that maps a non-negative integer `n` to a palindrome of length `n`. There is no need for unit tests here, your implementation can just be illustrated with a couple of examples, basically. Each of these examples, however, should be motivated.

Well, the question said no unit test, hmmm. BUT we really can implement a unit test quite simply! So, we will do just that:

```
(* unit test for our future make palindrome function *)
let test_make_palindrome make_palindrome =
  (* empty string: *)
  let b0 = (make_palindrome 0 = "")
  (* definition of palindrome *)
  and b1 = (let a = make_palindrome (Random.int 50)
             in a = string_reverse_mapi a)
  (* etc. *)
```



```
in b0 && b1;;
```

And there we have it! A unit test based on our previous `string_reverse_mapi` function! The palindrome generated should, of course, be the same when reversed!

Well, let's try thinking of a few different ways of generating palindromes.

We'll deal with the case where the length is an even number (as that is easier, slowly, but steadily).

One thing we could do is create a random string, `a`, and then reverse that string, `b`, and simply concatenate the two strings!

And so we have:

```
(* only works on even numbers *)
let make_palindrome_v0 n =
  let a = String.init (n/2) (fun _ -> char_of_int (int_of_char 'a' + Random.int 26))
  in (a ^ string_reverse_mapi a);;
```

Unfortunately, this wouldn't work on odd numbers! Harumph!

Well, I suppose we could have a separate condition when the length is odd such that we concatenate `a` and `b` (as mentioned earlier), but also concatenate a random character in between `a` and `b`!

And so we have:

```
(* works on odd and even numbers *)
let make_palindrome_v1 n =
  let a = String.init (n/2) (fun _ -> char_of_int (int_of_char 'a' + Random.int 26))
  in if (n mod 2) = 1
     then (a ^
           String.init 1 (fun _ -> char_of_int (int_of_char 'a' + Random.int 26)) ^
           string_reverse_mapi a)
     else (a ^ string_reverse_mapi a);;

(* testing function with unit test *)
let () = assert (test_make_palindrome make_palindrome_v1);;
```

And, of course, it passes our unit test.

Let's also look at a few examples, one with an odd number, one with an even number!

```
(* odd number *)
# make_palindrome_v1 7;;
- : string = "gxrrzrxg"

(* even number *)
# make_palindrome_v1 8;;
- : string = "bvzwwzvb"
```

Well, our current strategy seems quite wasteful! We create two strings, concatenate them (creating

another string) and finally return the concatenated string. In fact, in the case of an odd number, we create four strings (albeit one of them is just of length 1)!

The question really just asks for any palindrome, not a random one; so, I suppose we could try out something which with many conditions such that it generates a certain pattern.

A simple pattern to decide upon would be "abcba", or "abccba".

Well, in this case, we just need to iterate the alphabet upwards to begin with, and then downwards!

And so we have:

```
(* creates only one string (of length n) *)
let make_palindrome_v2 n =
  String.init
    n
    (fun i -> if i < (n/2)
      then char_of_int (int_of_char 'a' + i)
      else if i > (n/2) && n mod 2 = 0
      then char_of_int (int_of_char 'a' + i - 1 - (i mod (n/2)) * 2)
      else if i > (n/2)
      then char_of_int (int_of_char 'a' + i - 2 - (i mod ((n + 1)/2)) *
2)
      else if n mod 2 = 0
      then char_of_int (int_of_char 'a' + i - 1)
      else char_of_int (int_of_char 'a' + i));;

(* testing function with unit test *)
let () = assert (test_make_palindrome make_palindrome_v2);;
```

And, of course, the function passes our unit test.

And, looking at a few examples (one on an odd number, and one on an even number):

```
# make_palindrome_v2 5;;
- : string = "abcba"

# make_palindrome_v2 6;;
- : string = "abccba"
```

BUT wait a second, we can still randomise the middle char a bit more, if we are to make a string of an odd length; so why don't we try that?

```
(* creates slightly more random palindromes for odd numbers *)
let make_palindrome_v3 n =
  String.init
    n
    (fun i -> if i < (n/2)
      then char_of_int (int_of_char 'a' + i)
      else if i > (n/2) && n mod 2 = 0
      then char_of_int (int_of_char 'a' + i - 1 - (i mod (n/2)) * 2)
      else if i > (n/2)
      then char_of_int (int_of_char 'a' + i - 2 - (i mod ((n + 1)/2)) *
2)
      else if n mod 2 = 0
      then char_of_int (int_of_char 'a' + i - 1)
      else char_of_int (int_of_char 'a' + i));;
```

```

        else if n mod 2 = 0
        then char_of_int (int_of_char 'a' + i - 1)
        else char_of_int (int_of_char 'a' + Random.int 26));;

(* testing function with unit test *)
let () = assert (test_make_palindrome make_palindrome_v3);;

```

And, looking at a few examples (one on an odd number, and one on an even number):

```

# make_palindrome_v3 5;;
- : string = "abpba"

# make_palindrome_v3 6;;
- : string = "abccba"

```

BUT wait a second! There has been a much simpler solution all this while! The definition of a palindrome is only that it's reverse is the same as itself. Hence, if we create a string only consisting of one character, it'll be the same no matter how we jumble it up!

And so we have:

```

(* least intensive, in terms of space and calculations *)
let make_palindrome_v99 n =
  String.make n 'a';;

(* testing function with unit test *)
let () = assert (test_make_palindrome make_palindrome_v99);;

```

And it still fits the specifications of the question :-)

And, it passes the unit test too :-)

Not forgetting examples:

```

even number
# make_palindrome_v99 10;;
- : string = "aaaaaaaaaa"

odd number
# make_palindrome_v99 9;;
- : string = "aaaaaaaaa"

```

And, there we have version 99. Moving on!

4.9. question 07

Q. A palindrome detector is a function of type `string -> bool`, i.e., a predicate, such that applying it to a palindrome yields true whereas applying it to a string that is not a palindrome yields false

Let's first again begin with a unit test, perhaps using our previously defined `make_palindrome` function!

```

(* unit test for our future palindrome predicate *)
let test_palindromeep palindromep =
  (* empty string: *)

```

```
let b0 = (palindromep "")
  (* testing a palindrome *)
and b1 = (let a = make_palindrome_v3 (Random.int 50)
  in palindromep a)
in b0 && b1;;
```

Loki: Oh boy, something feels wrong here! We're headed in for trouble!
[name redacted]: what?

4.9.1. **implement a palindrome detector using String.mapi.**

Wait a second! We've done this earlier!

We reversed a string using String.mapi! Why don't we just test whether the reversed string (using String.mapi) is the same as the original string. That ways we use String.mapi to create our very own palindrome predicate!

And so we have:

```
(* using string_reverse_mapi (which uses String.mapi) *)
let palindromep_mapi s =
  string_reverse_mapi s = s;; (* :-) *)

(* alternatively, copy pasting the code *)
let palindromep_mapi_v1 s =
  let s_reversed = String.mapi
    (fun i _ -> let s_length = String.length s in
      String.get s (s_length - i - 1))
    s in
  s_reversed = s;; (* :-) :-) *)

(* testing function with unit test *)
let () = assert (test_palindromep palindromep_mapi_v1);;
```

YAY! Moving on swiftly, to use our creative juices elsewhere!

4.9.2. **implement a palindrome detector using recursion over a non-negative integer.**

Well, how do we test whether a palindrome is a palindrome? We could use our reverse function that we previously created using recursion.

BUT let's go a bit deeper.

Well, something simpler we could do is traverse the string forwards and traverse the string backwards, and check whether the characters at the front are the same as the characters at the end!

Hmmm. Let's not repeat our previous mistakes, and think about a base case and the inductive step.

Base case: $n = 0$

- we're done with all our testing! we can return true!

Inductive step: $n' = n - 1$

- hmmm. let's check whether this character from the front is the the character from the back, n' indices away both ways!

And, so we have:

```
(* using recursion *)
let palindromep_rec s =
  let s_length = String.length s in
  let rec visit n =
    if n = 0
    then true
    (* (String.get s 0) = (String.get s (s_length - 1)) *)
    else let n' = n - 1 in
         (String.get s n' = String.get s (s_length - n)) && visit n' in
    visit s_length;;

(* testing function with unit test *)
let () = assert (test_palindrome palindromep_rec);;
```

And, so it works! Our function has no bugs? Nothing? It passed the unit test, so it has to be working, right?

4.9.3. **elcome Mr. Dijkstra!**

W

Let's think about our unit test again. It only tests whether the palindrome predicate returns true. Hence, a function that always returns true will also pass!

```
(* however, the following function will also pass the unit test! *)
let palindromep_fake s =
  true;;

(* testing function with unit test *)
let () = assert (test_palindrome palindromep_fake);;
```

Oh no! Loki was right! We have been outsmarted... by ourselves!

To quote Dijkstra:

```
"Program testing can be used to show the presence of bugs, but never to show
their absence!"
- Edsger Wybe Dijkstra
```

And a wise quote that is! When something passes our unit test, we only know that it worked for those particular test cases. And, so, we can't know whether there is a true absence of bugs, because we simply haven't tested all the cases!

For instance, in the following case:

```
(* unlike the following example, where we can have complete code coverage *)
let test_not not_function =
  let b0 = (not_function true = false)
  and b1 = (not_function false = true)
  in b0 && b1;;

let () = assert (test_not not);;
```

Hence, since there is full code coverage here, we cannot write a "fake" function as the "fake" function will have to fulfill all the criteria needed to implement the real function.

Hence, the "fake" function will actually be the same as the real function!

The more cases we test, the greater code coverage we have! If we test all possible cases, then we can truly say that our program is free of bugs, as we would have complete code coverage!

So, at the very least, let's at least increase our code coverage, so we can have a more comprehensive test (even though it can never be fully comprehensive):

```
(* hence, we can improve our unit test to increase code coverage *)
let test_palindrome_v1 palindrome =
  (* empty string: *)
  let b0 = (palindrome "")
  (* testing a palindrome *)
  and b1 = (let a = make_palindrome_v3 (Random.int 50)
          in palindrome a)
  (* testing a string that is not a palindrome *)
  and b2 = (let not_palindrome =
          ("a" ^
           (String.init
            (Random.int 50)
            (fun _ -> char_of_int (int_of_char 'a' + Random.int 26)))) ^
           "b")
          in not (palindrome not_palindrome))
  in b0 && b1 && b2;;

(* testing previous functions with unit test *)
let () = assert (test_palindrome_v1 palindrome_map);;
let () = assert (test_palindrome_v1 palindrome_map_v1);;
let () = assert (test_palindrome_v1 palindrome_rec);;
(* and they still work! *)

(* let () = assert (test_palindrome_v1 palindrome_fake);; *)
(* and of course, our fake function gets caught! *)
```

And, so we were able to catch the fake function! Another way we could write a fake_{function} is one where we implement the palindrome predicate correctly upto strings of length 50, and post that, we can do whatever we want within our function. And, so, we see we still don't quite have complete code coverage;

And, the unfortunate reality is, it's quite impractical to have complete code coverage in cases like these. So we strive for the best we can do!

4.10.

q

uestion 08

Q. Implement an OCaml function that reverses a palindrome.

Sigh. Fair enough.

Well, we know that a palindrome is the same as its reverse; hence, we can simply use the identity function.

```
(* Question 8 (not optional) *)  
  
let reverse_palindrome s =  
  s;; (* :-) *)
```

Not bad, not bad indeed!

4.11.

q

uestion 10

Q. Implement String.map using String.mapi.

Well, we've already done something similar! In a function, we can simply ignore any arguments we don't want to use! And that's exactly what we'll do:

```
let string_map f s =  
  String.mapi  
    (fun _ c -> f c)  
    s;;  
  
(* a function to test our new string_map function *)  
let test_string_map f s =  
  (string_map f s = String.map f s) ;;  
  
(* testing function with testing function *)  
let () = assert (test_string_map  
  (fun c -> char_of_int (int_of_char c + 1))  
  "give me a string, any string!") ;;  
  
(* testing function with testing function and an empty string *)  
let () = assert (test_string_map  
  (fun c -> char_of_int (int_of_char c + 1))  
  "") ;;
```

And, of course, the function passes our mini test as well!

4.12.

q

uestion 11

Well, this one is ever so slightly tougher!
BUT still not something we can't do!

String.init allows us to use a function. String.mapi uses a function and applies it to each of the characters of the string that String.mapi is applied to.

We can simply use String.init's function to initialise the string in such a manner where we use the function provided to us (the one that String.mapi is supposed to use).

And, so we have:

```
let string_mapi f s =
  String.init
    (String.length s)
    (fun i -> f i (String.get s i));;

(* a function to test our new string_map function *)
let test_string_mapi f s =
  (string_mapi f s = String.mapi f s);;

(* testing function with testing function *)
let () = assert (test_string_mapi
  (fun i c -> char_of_int (int_of_char c + i))
  "abcdef");;

(* testing function with testing function and an empty string *)
let () = assert (test_string_mapi
  (fun i c -> char_of_int (int_of_char c + i))
  "");;
```

And, of course, the function passes our mini test as well!

4.13.

onclusion

c

And so we were able to complete our strings mini-project! We were able to create our own version of functions that come with standard OCaml libraries; functions such as the concatenation function, String.map and String.mapi. Further, we were also able to be the boss of OCaml and tell it do things our way; that is, we were able to run evaluations upwards or downwards, and we also learnt what it means to go upwards or downwards. Further, we created our very own version of nat_parafold_right:nat_parafold_right_down! One that evaluates downwards rather than upwards! We saw how we can use these two to implement reversing a string in different directions. We also learnt how we can ignore an argument in a function when we implemented String.map using String.mapi. We also learnt how we may be able to implemnt our own version of String.mapi using String.init.

5. cracking polynomial functions

5.1. introduction

We've all learnt elementary algebra in high school, and that is exactly the skill we'll require here! We'll essentially be solving a system of equations; through solving a system of equations, we'll be ale to crack polynomials of degree 1, 2, 3, 4 and 5. By crack, of course, I mean we'll be able to find the coefficients of the polynomial given the function that implements the polynomial! And, oh, I completely forgot; also degree 0 polynomials, whatever that means? Yes, we'll explore what exactly it means for a function to implement a polynomial of degree 0 and what kind of function that'd be;

let's not be strict and try and have an open (or non-strict, if you prefer) mind! Finally, we also learn the importance of helpful error messages.

5.2. first degree

From Professor Olivier Danvy's lecture notes:

Given two coefficients a_1 and a_0 , the following function returns a polynomial function of degree 1:

```
let make_polynomial_1 a1 a0 x =  
  a1 * x + a0;;
```

Given such a polynomial function of degree 1, we want to compute its coefficients. So your task, should you choose to accept it, but hey, it's not like this mission is impossible, witness the solution just below, so there really is no reason to digress here, is to implement a function `crack1` of type:

```
(int -> int) -> int * int
```

that passes the following unit test:

```
let test_crack_1_once candidate =  
  let a1 = random_int ()  
  and a0 = random_int ()  
  in let p1 = make_polynomial_1 a1 a0  
     in let expected_result = (a1, a0)  
        and ((a1', a0') as actual_result) = candidate p1  
        in if actual_result = expected_result  
           then true  
           else let () = Printf.printf  
                  "test_crack_1_once failed for %i * x^1 + %i with  
(%i, %i)\n"  
                  a1 a0 a1' a0'  
                in false;;
```

In words: to test a candidate cracking function,

- we manufacture a first-degree polynomial function using two random integers;
- we apply the candidate cracking function to this polynomial function;
- if the resulting two integers are the same as the two random integers, the candidate cracking function passes this test;
- otherwise, the candidate cracking function does not pass this test and an error message is emitted.

This being said, by Professor Olivier Danvy, we can move on to our own functions!

For each subsequent degree, our function will accept a function that itself accepts one more integer, and return a tuple with just one more integer as well. The additional integer in the tuple will

represent the additional coefficient.

And, so, ready, set, go!

For our first cracking function we can just calculate a_1 (i.e. the coefficient of x^1) by subtracting p_1_0 !

And, so we have:

```
let test_crack_1 candidate =
  test_crack_1_once candidate && test_crack_1_once candidate &&
  test_crack_1_once candidate;;

let crack_1 p1 =
  let a1 = p1 1 - p1 0
  and a0 = p1 0
  in (a1, a0);;

let () = assert (test_crack_1 crack_1);;

let crack_1_v1 p1 =
  let a0 = p1 0
  in let a1 = p1 1 - a0
  in (a1, a0);;
(* with this, we only have to calculate p1 0 once *)

let () = assert (test_crack_1 crack_1_v1);;
```

We also saw another implementation where there are lesser calculations but more space is used!

1, cracked.

5.3. *second degree*

New degree, same story!

Let's start with our function that makes the polynomial:

```
let make_polynomial_2 a2 a1 a0 x =
  a2 * x * x + a1 * x + a0;;
```

Next, we can write our unit test:

```
let test_crack_2_once candidate =
  let a2 = random_int ()
  and a1 = random_int ()
  and a0 = random_int ()
  in let p2 = make_polynomial_2 a2 a1 a0
  in let expected_result = (a2, a1, a0)
  and ((a2', a1', a0') as actual_result) = candidate p2
  in if actual_result = expected_result
  then true
  else let () = Printf.printf
```

```

"%s\ntest_crack_2_once failed for %i * x^2 + %i * x^1
+ %i\nwith %i instead of %i, %i instead of %i, and %i instead of %i\n"
(Array.get commiseration (Random.int (Array.length
commiseration)))
a2 a1 a0 a2' a2 a1' a1 a0' a0
in false;;

let test_crack_2 candidate =
  test_crack_2_once candidate && test_crack_2_once candidate &&
test_crack_2_once candidate;;

```

Finally, the crack(alack)ing function itself:

```

let crack_2 p2 =
  let a0 = p2 0
  in let a2_add_a1 = p2 1 - a0
    and a2_sub_a1 = p2 ~-1 - a0
    in let a2 = (a2_add_a1 + a2_sub_a1) / 2
      in let a1 = a2_add_a1 - a2
      in (a2, a1, a0);;

let () = assert (test_crack_2 crack_2);;

```

And, there we have it! We were able to solve a system of equations. It's elementary, Watson!

5.4. zeroth degree

New (old?) degree, same story!

Let's start with our function that makes the polynomial:

```

let make_polynomial_0 a0 x =
  a0;;

```

Well, we can just straight away return a0. We've seen this pattern before; more on that later!

Next, we can write our unit test:

```

let test_crack_0_once candidate =
  let a0 = random_int ()
  in let p0 = make_polynomial_0 a0
    in let expected_result = a0
      and (a0' as actual_result) = candidate p0
      in if actual_result = expected_result
        then true
        else let () = Printf.printf
          "%s\ntest_crack_0_once failed for %i\nwith %i instead
of %i\n"
          (Array.get commiseration (Random.int (Array.length
commiseration)))
          a0 a0' a0'
        in false;;

let test_crack_0 candidate =
  test_crack_0_once candidate && test_crack_0_once candidate &&
test_crack_0_once candidate;;

```

Okay, that was pretty standard.

Finally, the crack(alack)ing function itself! Well, we could just apply the function to 0? Let's try that first!

```
let crack_0 p0 =  
  p0 0;;  
  
(* and it works! *)  
let () = assert (test_crack_0 crack_0) ;;
```

AND it works!

However, one quickly realises that p0 is a non-strict function! Hence, we may apply the function to any argument and we'll get the same result! So why 0, when we can do so much more!

```
(* so different implementations like these: *)  
let crack_0_v1 p0 =  
  p0 "this is a secret message no one will ever see (apart from you)!";;  
  
let crack_0_v2 p0 =  
  p0 'a';;  
  
let crack_0_v3 p0 =  
  p0 (random_int ());;  
  
let crack_0_v4 p0 =  
  p0 "";;  
  
let crack_0_v5 p0 =  
  p0 ' ';;  
  
let crack_0_v6 p0 =  
  p0 false;;  
  
(* work too! *)  
let () = assert (test_crack_0 crack_0_v1) ;;  
let () = assert (test_crack_0 crack_0_v2) ;;  
let () = assert (test_crack_0 crack_0_v3) ;;  
let () = assert (test_crack_0 crack_0_v4) ;;  
let () = assert (test_crack_0 crack_0_v5) ;;  
let () = assert (test_crack_0 crack_0_v6) ;;  
  
(* even passing in a unit works just fine! *)  
let crack_0_v7 p0 =  
  p0 () ;;  
  
let () = assert (test_crack_0 crack_0_v7) ;;
```

The magic of non-strict functions!

Looks quite similar to defining a variable! Instead, we predefine a computation that we can run any time!

5.5. *third degree*

New degree, same story!

Let's start with our function that makes the polynomial:

```
let make_polynomial_3 a3 a2 a1 a0 x =  
  a3 * x * x * x + a2 * x * x + a1 * x + a0;;
```

Next, we can write our unit test:

```
let test_crack_3_once candidate =  
  let a3 = random_int ()  
  and a2 = random_int ()  
  and a1 = random_int ()  
  and a0 = random_int ()  
  in let p3 = make_polynomial_3 a3 a2 a1 a0  
     in let expected_result = (a3, a2, a1, a0)  
        and ((a3', a2', a1', a0') as actual_result) = candidate p3  
        in if actual_result = expected_result  
           then true  
           else let () = Printf.printf  
              "%s\ntest_crack_3_once failed for %i * x^3 + %i * x^2  
+ %i * x^1 + %i\n\nwith %i instead of %i, %i instead of %i, %i instead of %i,  
and %i instead of %i\n"  
              (Array.get commiseration (Random.int (Array.length  
commiseration)))  
              a3 a2 a1 a0 a3' a3 a2' a2 a1' a1 a0' a0  
              in false;;  
  
let test_crack_3 candidate =  
  test_crack_3_once candidate && test_crack_3_once candidate &&  
test_crack_3_once candidate;;
```

Finally, the crack(alack)ing function itself:

```
let crack_3 p3 =  
  let a0 = p3 0  
  in let a3_add_a2_add_a1 = p3 1 - a0  
     and min_a3_add_a2_sub_a1 = p3 ~-1 - a0  
     in let a2 = (a3_add_a2_add_a1 + min_a3_add_a2_sub_a1) / 2  
        in let a3 = (p3 2 - (2 * a3_add_a2_add_a1) - (2 * a2) - a0) / 6  
           in let a1 = a3_add_a2_add_a1 - a3 - a2  
           in (a3, a2, a1, a0);;  
  
let () = assert (test_crack_3 crack_3);;
```

And, there we have it! We were able to solve a system of equations. It's elementary, Watson!

5.6. *fourth degree*

New degree, same story!

Let's start with our function that makes the polynomial!

For degree 4 onward polynomials, I decided to use horner's rule because it is more computationally

efficient as it has lesser multiplication operations; behold:

```
let make_polynomial_4 a4 a3 a2 a1 a0 x =  
  a0 + x * (a1 + x * (a2 + x * (a3 + x * a4)));;
```

Next, we can write our unit test:

```
let test_crack_4_once candidate =  
  let a4 = random_int ()  
  and a3 = random_int ()  
  and a2 = random_int ()  
  and a1 = random_int ()  
  and a0 = random_int ()  
  in let p4 = make_polynomial_4 a4 a3 a2 a1 a0  
     in let expected_result = (a4, a3, a2, a1, a0)  
        and ((a4', a3', a2', a1', a0') as actual_result) = candidate p4  
        in if actual_result = expected_result  
           then true  
           else let () = Printf.printf  
                  "%s\ntest_crack_5_once failed for %i * x^4 + %i * x^3  
+ %i * x^2 + %i * x^1 + %i\nwith %i instead of %i, %i instead of %i, %i instead  
of %i, %i instead of %i, and %i instead of %i\n"  
                  (Array.get commiseration (Random.int (Array.length  
commiseration)))  
                  a4 a3 a2 a1 a0 a4' a4 a3' a3 a2' a2 a1' a1 a0' a0  
                in false;;  
  
let test_crack_4 candidate =  
  test_crack_4_once candidate && test_crack_4_once candidate &&  
test_crack_4_once candidate;;
```

Finally, the crack(alack)ing function itself:

```
let () = assert (test_crack_4 crack_4);;
```

And, there we have it! We were able to solve a system of equations. It's elementary, Watson!

5.7. fifth degree

Why stop at 4? We know it scales, so here's 5!

New degree, same story!

Let's start with our function that makes the polynomial:

```
let make_polynomial_5 a5 a4 a3 a2 a1 a0 x =  
  a0 + x * (a1 + x * (a2 + x * (a3 + x * (a4 + x * a5))));;
```

Next, we can write our unit test:

```
let test_crack_5_once candidate =  
  let a5 = random_int ()  
  and a4 = random_int ()  
  and a3 = random_int ()  
  and a2 = random_int ()
```

```

and a1 = random_int ()
and a0 = random_int ()
in let p5 = make_polynomial_5 a5 a4 a3 a2 a1 a0
  in let expected_result = (a5, a4, a3, a2, a1, a0)
    and ((a5', a4', a3', a2', a1', a0') as actual_result) = candidate p5
    in if actual_result = expected_result
      then true
      else let () = Printf.printf
        "%s\ntest_crack_5_once failed for %i^5 + %i * x^4
+ %i * x^3 + %i * x^2 + %i * x^1 + %i\nwith %i instead of %i, %i instead
of %i, %i instead of %i, %i instead of %i, %i instead of %i, and %i instead
of %i\n"
        (Array.get commiseration (Random.int (Array.length
commiseration)))
        a5 a4 a3 a2 a1 a0 a5' a5 a4' a4 a3' a3 a2' a2 a1' a1
a0' a0
        in false;;

let test_crack_5 candidate =
  test_crack_5_once candidate && test_crack_5_once candidate &&
test_crack_5_once candidate;;

```

Finally, the crack(alack)ing function itself:

```

let test_crack_5 candidate =
  test_crack_5_once candidate && test_crack_5_once candidate &&
test_crack_5_once candidate;;

let crack_5 p5 =
  let a0 = p5 0
  in let a5_add_a4_add_a3_add_a2_add_a1 = p5 1 - a0
    and min_a5_add_a4_sub_a3_add_a2_sub_a1 = p5 ~-1 - a0
    in let a4_add_a2 = (a5_add_a4_add_a3_add_a2_add_a1 +
min_a5_add_a4_sub_a3_add_a2_sub_a1) / 2
      and e_36a5_add_16a4_add_8a3_add_4a2_add_2a1 = p5 2 - a0
      and e_min_36a5_add_16a4_sub_8a3_add_4a2_sub_2a1 = p5 ~-2 - a0
      in let e_32a5_add_12a4_add_8a3_add_2a1 =
e_36a5_add_16a4_add_8a3_add_4a2_add_2a1 - 4 * a4_add_a2
        and e_min_32a5_add_12a4_sub_8a3_sub_2a1 =
e_min_36a5_add_16a4_sub_8a3_add_4a2_sub_2a1 - 4 * a4_add_a2
        in let a4 = (e_32a5_add_12a4_add_8a3_add_2a1 +
e_min_32a5_add_12a4_sub_8a3_sub_2a1) / 24
          in let a2 = a4_add_a2 - a4
            in let e_243a5_add_81a4_add_27a3_add_9a2_add_3a1 = p5 3 - a0
              in let e_240a5_add_24a3 =
e_243a5_add_81a4_add_27a3_add_9a2_add_3a1 - 3 * a5_add_a4_add_a3_add_a2_add_a1
- 78 * a4 - 6 * a2
                and e_195a5_add_15a3 =
(e_243a5_add_81a4_add_27a3_add_9a2_add_3a1 - 81 * a4 - 9 * a2) - 3 *
((e_32a5_add_12a4_add_8a3_add_2a1 - 12 * a4) / 2)
                in let a5 = (8 * e_195a5_add_15a3 - 5 *
e_240a5_add_24a3) / 360
                  in let a3 = (e_195a5_add_15a3 - 195 * a5) / 15
                    in let a1 = a5_add_a4_add_a3_add_a2_add_a1 - a5 -
a4 - a3 - a2
                      in (a5, a4, a3, a2, a1, a0);;

let () = assert (test_crack_5 crack_5);;

```

YIKES! That was some system of equation, alright!

BUT, there we have it! We were able to solve a system of equations. It's elementary, Watson!

5.8. *higher degree to lower degree*

One final thing; we can use any polynomial cracking function to crack a polynomial of a lower degree!

That is to say, a "degree 5 cracker" can also crack polynomials of degree 4, 3, 2, 1, and 0. Similarly, a "degree 4 cracker" can also crack polynomials of degree 3, 2, 1, and 0.

Hence, there is no reason to ask individually whether a "degree 3 cracker" can crack a polynomial of degree 2 or 1, as we can see from the above rule that a higher degree cracker can always crack a lower degree polynomial.

We do something similar when we implement `nat_parafold_right` in terms of `nat_fold_right`: we ignore one of the values.

Like so:

```
(* using crack_5 to implement crack_0 *)
let crack_0_using_crack_5 p0 =
  let (a5, a4, a3, a2, a1, a0) = crack_5 p0
  in a0;;

let () = assert (test_crack_0 crack_0_using_crack_5);;
```

The value of `a5`, `a4`, `a3`, and `a2` will simply be 0. Hence, we can simply throw those values out and only return the relevant ones.

Similarly, for `crack_1`, `crack_2`, `crack_3`, `crack_4`:

```
(* using crack_5 to implement crack_1 *)
let crack_1_using_crack_5 p1 =
  let (a5, a4, a3, a2, a1, a0) = crack_5 p1
  in (a1, a0);;

let () = assert (test_crack_1 crack_1_using_crack_5);;

(* using crack_5 to implement crack_2 *)
let crack_2_using_crack_5 p2 =
  let (a5, a4, a3, a2, a1, a0) = crack_5 p2
  in (a2, a1, a0);;

let () = assert (test_crack_2 crack_2_using_crack_5);;

(* using crack_5 to implement crack_3 *)
let crack_3_using_crack_5 p3 =
  let (a5, a4, a3, a2, a1, a0) = crack_5 p3
  in (a3, a2, a1, a0);;

let () = assert (test_crack_3 crack_3_using_crack_5);;

(* using crack_5 to implement crack_4 *)
```



```
let crack_4_using_crack_5 p4 =
  let (a5, a4, a3, a2, a1, a0) = crack_5 p4
  in (a4, a3, a2, a1, a0);;

let () = assert (test_crack_4 crack_4_using_crack_5);;
```

So, there we have a hard and fast rule to answer the many subsidiary questions, all at once; once again, many birds with one stone!

5.9. conclusion

And there we have it! A few overall observations to begin with. Although we were not able to generalise the elephant that is cracking polynomial functions just yet (maybe sometime in the future?), we realised that to crack a polynomial of degree n , we needed, at the very least $n+1$ values! Another thing we learnt slightly more of was how to ignore values; that is, we learnt how we can implement a lower degree cracker using a higher degree cracker!

6. conclusion

That's all folks! Some journey that was, but we finally landed on moon and took our first step! We learnt about OCaml inside-out which helped us choose the direction of evaluation rather than having OCaml choose it for us! We looked at how `nat_fold_right` and `nat_parafold_right` are really the same function, and we learnt even more about this when we studied the underlying determinism of OCaml and saw that curried functions are the same as uncurried ones; that is, they lead to the same computation. Moreover, we learnt what it means for two expressions to be observationally equivalent. We went back to high school and cracked a bunch of polynomials by solving a system of equations; simultaneously, we learnt how to create helpful error messages that can make debugging a breeze! We also played around with strings; literally, by reversing them! We learnt the different ways we could make palindrome, and how we would test making those palindromes. More importantly, we also learnt about code coverage and why unit tests cannot tell us whether our programs are free from bugs (although, they can still be helpful to test their presence).

Overall, we came out with a much deeper understanding of OCaml as a whole, and we're now able to make sense of why our programs run in a certain way, and why they cause errors in a certain way.

```
T.T.T.
- Piet Hein
```

And, things, really do take time; but, when you put in that time, they turn out to be wonderful!